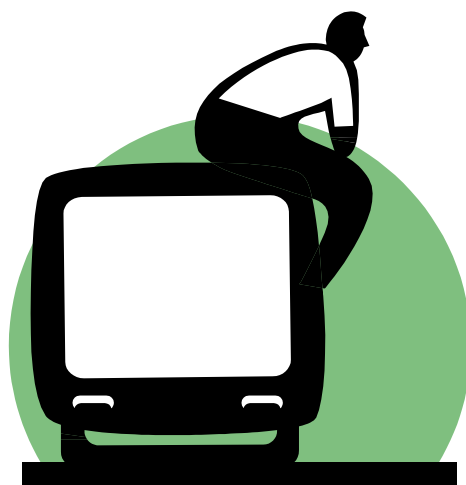


Linguagem de Programação - C

Sônia Virginia Alves França



2009/01

Índice

Capítulo 1- Introdução ao C	5
1.1 Linguagens de Programação	5
1.2 Linguagens de baixo e alto nível.....	5
1.3 Histórico do C.....	6
1.4 Criação, compilação e execução de um programa.....	7
1.4.1 Criação do programa	7
1.4.2 Compilação.....	7
1.4.3 Execução do programa.....	7
1.5 O modelo de compilação da linguagem C	8
1.5.1 O pré-processador	8
1.5.2 O compilador	9
1.5.3 O assembler.....	9
1.5.4 O linker	9
1.5.5 A utilização de bibliotecas	9
Capítulo 2 - Técnicas Básicas de Programação	10
2.1 Tipos de dados primitivos	10
2.2 Identificadores	10
2.3 Variáveis e Constantes.....	11
2.3.1 Declaração de Variáveis	11
2.3.2 Declaração de Constantes	11
2.4 Operações Básicas.....	12
2.4.1 Operador de Atribuição	12
2.4.2 Operadores Aritméticos	12
2.4.3 Operações Relacionais	14
2.4.4 Operadores Lógicos	15
2.5 Atribuições reduzidas	15
2.6 Operadores pré e pós-fixados	16
2.7 Conversão de tipo (Casting).....	16
2.8 Comandos de Entrada e Saída de Dados	17
2.8.1 Entrada de Dados.....	17
2.8.2 Saída de Dados.....	17
2.9 Palavras Reservadas do C.....	18

2.10 Estrutura de um programa básico em C.....	19
2.10.1 Comentários	19
2.11 Conhecendo o Dev-C++	21
Exercícios Propostos I	21
<i>Capítulo 3 - Comandos de Seleção.....</i>	<i>27</i>
3.1 Introdução	27
3.2 IF...ELSE (SE...SENÃO).....	27
3.3 SWITCH (Escolha)	29
Exercícios Propostos II.....	31
<i>Capítulo 4 - Comandos de Repetição</i>	<i>35</i>
4.1 Introdução	35
4.2 FOR (PARA).....	35
4.3 WHILE (ENQUANTO – teste no início)	36
4.4 DO...WHILE (Faça...enquanto - teste no final)	37
4.5 Comandos Desestruturadores	39
Exercícios Propostos III	39
<i>Capítulo 5 – Vetores e Registros</i>	<i>44</i>
5.1 Vetores - Tipos de Dados Estruturados	44
5.2 Registros	46
5.3 Vetor de Registros	47
Exercícios Propostos IV.....	48
<i>Capítulo 6 – Modularização</i>	<i>55</i>
6.1 Introdução	55
6.2 Declarando uma função	55
6.3 Variáveis Locais.....	56
6.4 Chamando Funções.....	56
6.5 Comando return.....	56
6.6 Argumentos.....	57
Exercícios Propostos V	58
<i>Capítulo 7 – Manipulação de Caracteres</i>	<i>61</i>
7.1 Funções de manipulação de cadeia de caracteres: biblioteca string.h	61
7.2 Funções de conversão de caractere: biblioteca ctype.h.....	63
7.3 Funções de conversão: biblioteca stdlib.h	63
7.4 Funções de teste para caracteres: Biblioteca ctype.h	65
7.5 Validação de Campo.....	66
Exercícios Propostos VI.....	67

Capítulo 8 – Comandos de tela	69
8.1 Comandos Básicos de Tela	69
Exercícios Propostos VII	73
Capítulo 9 – Arquivo	75
9.1 Alocação Dinâmica - Ponteiros	75
9.2 Arquivos	77
9.2.1 Declaração de um Arquivo	77
9.2.2 Abrindo e Fechando um Arquivo	77
9.2.3 Ler e Gravar Informação no Arquivo	79
9.2.4 Acesso Randômico	79
Exercícios Propostos VIII	84
Capítulo 10 – Extras	85
10.1 Tabela ASCII	85
10.2 Acentos	86
10.3 Molduras	88
10.4 Biblioteca time.h	90
Capítulo 11 – Boas Práticas de Programação	91
11.1 Qualidades de um Programa	91
11.2 Formas de Melhoria da Eficiência de um Programa	92

Capítulo 1- Introdução ao C

1.1 Linguagens de Programação

Um *programa* de computador é um conjunto *instruções* que representam um *algoritmo* para a resolução de algum problema. Estas instruções são *escritas* através de um conjunto de *códigos* (símbolos e palavras). Este conjunto de códigos possui regras de estruturação lógica e sintática própria. Dizemos que este conjunto de símbolos e regras formam uma *linguagem de programação*.

1.2 Linguagens de baixo e alto nível

Podemos dividir, genericamente, as linguagens de programação em dois grandes grupos: as linguagens de *baixo nível* e as de *alto nível*.

- **Linguagens de baixo nível:** São linguagens voltadas para a máquina, isto é, são escritas usando as instruções do microprocessador do computador. São genericamente chamadas de linguagens *Assembly*. *Vantagens:* os programas são executados com maior *velocidade* de processamento e ocupam menos *espaço* na memória. *Desvantagens:* em geral, os programas em Assembly têm pouca *portabilidade* (um código gerado para um tipo de processador não serve para outro) e não são estruturados, tornando a programação mais difícil.
- **Linguagens de alto nível:** São linguagens voltadas para o ser humano. Em geral utilizam sintaxe estruturada tornando seu código mais legível. Necessitam de *compiladores* ou *interpretadores* para gerar instruções do microprocessador. Interpretadores fazem a interpretação de *cada* instrução do programa fonte executando-a dentro de um ambiente de programação, **Basic** e **AutoLISP**, por exemplo. Compiladores fazem a tradução de todas as instruções do programa fonte gerando um programa executável. Estes programas executáveis (*.exe) podem ser executados fora dos ambientes de programação, **C** e **Pascal** por exemplo. As linguagens de alto nível podem se distinguir quanto a sua aplicação em *genéricas* como **C**, **Pascal** e **Basic** ou *específicas* como **Fortran** (cálculo matemático), **GPSS** (simulação), **LISP** (inteligência artificial) ou **CLIPPER** (banco de dados). *Vantagens:* por serem compiladas ou interpretadas, tem *maior*

portabilidade podendo ser executados em varias plataformas com pouquíssimas modificações. Em geral, a programação torna-se mais fácil por causa do maior ou menor grau de estruturação de suas linguagens. *Desvantagens:* em geral, as rotinas geradas são mais genéricas (e mais complexas) e por isso são mais lentas e ocupam mais memória.

1.3 Histórico do C

Desenvolvida nos laboratórios Bell na década de 70, a partir da Linguagem B (criada no final dos anos 60 por Ken Thompson), que foi reformulada por Brian Kernighan e Dennis M. Ritchie e posteriormente renomeada para C.

Podendo ser considerada uma linguagem de médio nível, pois possui instruções que a tornam ora uma linguagem de alto nível e estruturada como o Pascal, se assim se fizer necessário, ora uma linguagem de baixo nível pois possui instruções tão próximas da máquina, que só o Assembler possui.

De fato, com a linguagem C podemos construir programas organizados e concisos (como o Pascal), ocupando pouco espaço de memória com alta velocidade de execução (como o Assembler). Infelizmente, dada toda a flexibilidade da linguagem, também poderemos escrever programas desorganizados e difíceis de serem compreendidos (como usualmente são os programas em BASIC).

Devemos lembrar que a linguagem C foi desenvolvida a partir da necessidade de se escrever programas que utilizassem recursos próprios da linguagem de máquina de uma forma mais simples e portátil que o Assembler.

Existem inúmeras razões para a escolha da linguagem C como a predileta para os desenvolvedores “profissionais”. As características da Linguagem C servirão para mostrar o porquê de sua ampla utilização.

Características da Linguagem C:

- Portabilidade entre máquinas e sistemas operacionais;
- Dados compostos em forma estruturada;
- Programas estruturados;
- Total interação com o sistema operacional;
- Código compacto e rápido, quando comparado ao código de outras linguagens.

1.4 Criação, compilação e execução de um programa

Os estados de desenvolvimento de um programa em C são:

- Criação do programa fonte (texto);
- Compilação desse programa, para a sua tradução para código executável;
- Execução do código produzido;

No caso da detecção de erro em qualquer um dos estados, todos eles deverão ser repetidos desde o início.

1.4.1 Criação do programa

A criação dos programas fonte em linguagem C faz-se com o auxílio de um editor de texto genérico ou específico de um ambiente de desenvolvimento. Em geral, os arquivos de texto produzidos deverão ter a extensão .c, para poderem ser reconhecidos automaticamente pelo compilador como sendo arquivos contendo código fonte em C. Obviamente, o conteúdo dos arquivos deverá obedecer rigorosamente à sintaxe da linguagem C.

1.4.2 Compilação

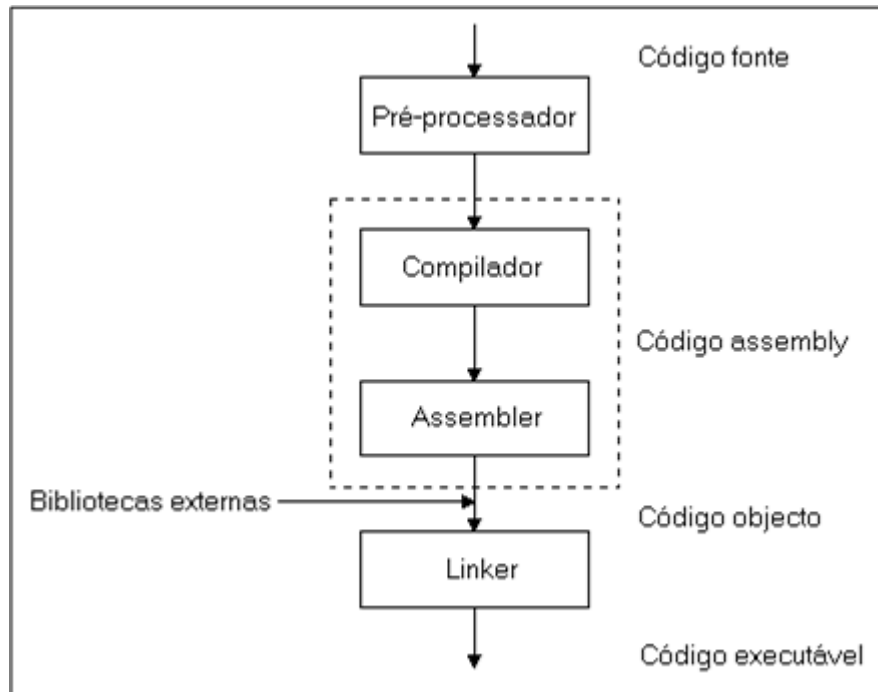
A compilação dos programas em C faz-se através da invocação de um compilador. Se existirem erros de sintaxe no código fonte, o compilador irá detectá-los e indicará a sua localização junto com uma breve descrição do erro. Erros na lógica do programa apenas poderão ser detectados durante a execução do mesmo. Se o programa não contiver erros de sintaxe o compilador produzirá código executável.

1.4.3 Execução do programa

Se a operação anterior tiver sucesso, a execução do programa compilado produzido faz-se simplesmente invocando-o como se fosse um comando do sistema operacional. Durante a execução podem tornar-se evidentes mais alguns erros: de execução (por exemplo: divisão por zero) ou erros que levam a um comportamento inesperado. Neste caso é necessário voltar à edição do programa fonte para corrigir a sua lógica, e depois efetuar também uma nova compilação para produzir a nova versão do código executável.

1.5 O modelo de compilação da linguagem C

Serão destacados apenas os pontos principais do modelo de compilação da linguagem C. Esse modelo pode ser ilustrado através da figura a seguir.



1.5.1 O pré-processador

O pré-processador atua apenas a nível de código fonte, modificando-o. Trabalha apenas com texto. Algumas das suas funções são:

- remover os comentários de um programa;
- interpretar diretivas especiais a si dirigidas, que começam pelo carácter #.

Por exemplo:

- `#include` - insere o conteúdo de um arquivo de texto no arquivo corrente. Esses arquivos são chamados de cabeçalhos (*header files*) e têm a extensão `.h`:

`#include <math.h>` - Insere o conteúdo do arquivo `math.h` com a declaração das funções matemáticas da biblioteca padrão.

- `#define` - define um nome simbólico cujas ocorrências no arquivo serão substituídas por outro nome ou constante (valores):

`#define MAX_ARRAY_SIZE 100` - substitui todas as ocorrências de `MAX_ARRAY_SIZE` por 100.

1.5.2 O compilador

Alguns compiladores traduzem o código fonte (texto) recebido do pré-processador para linguagem assembly (também texto). No entanto, é comum compiladores capazes de gerarem diretamente o código objeto (instruções do processador já em código binário).

1.5.3 O assembler

O assembler traduz código em linguagem assembly (texto) para código objeto. Pode estar integrado no compilador. O código objeto é geralmente armazenado em arquivos com a extensão .o (unix) ou .obj (ms-dos).

1.5.4 O linker

Se o programa referencia funções da biblioteca padrão ou outras funções contidas em arquivos com código fonte diferentes do principal (que contém a função `main()`), o linker combina todos os objetos com o resultado compilado dessas funções num único arquivo com código executável. As referências a variáveis globais externas também são resolvidas pelo linker.

1.5.5 A utilização de bibliotecas

A linguagem C é muito compacta. Muitas das funções que fazem parte de outras linguagens não estão diretamente incluídas na linguagem C. Temos como exemplo as operações de entrada/saída, a manipulação de strings e certas operações matemáticas.

A funcionalidade correspondente a estas e outras operações não faz parte integrante da linguagem, mas está incluída numa biblioteca externa, bastante rica. Todas essas operações são executadas através da invocação de funções externas definidas nessa biblioteca padrão.

Qualquer programador poderá desenvolver a sua própria biblioteca de funções, podendo até substituir algumas das funções padrão, e também utilizar outras bibliotecas comerciais já existentes.

Capítulo 2 - Técnicas Básicas de Programação

2.1 Tipos de dados primitivos

Dados são representados pelas informações a serem tratadas (processadas) por um computador.

Inteiro: toda e qualquer informação numérica que pertença ao conjunto dos números inteiros relativos (negativa, nula ou positiva).

Tipo	Tamanho	Valores
short	16 bits	-32.768 a +32.767
int	32 bits	-2.147.483.648 a + 2.147.483.647
long	64 bits	-9.223.372.036.854.775.808 a +9.223.372.036.854.775.807

Real: Toda e qualquer informação numérica que pertença ao conjunto de números reais (negativa, positiva ou nula).

Tipo	Tamanho	Valores
float	32 bits	3.4E-38 a 3.4E+38
double	64 bits	1.7E-308 a 1.7E+308

Caractere (char): Toda e qualquer informação composta por um caractere alfa numérico (a, b, c,...z, A, B, C,...Z, 0...9) e /ou especiais (ex; #?@!<?). O caractere deve ser indicado entre apóstrofo ‘ ’.

Lógico: O C não tem o tipo lógico.

2.2 Identificadores

Representam os nomes escolhidos para rotular as variáveis, constantes, procedimentos e funções.

Regras:

1. primeiro caractere deve ser uma letra;
2. Os nomes devem ser formados por caracteres pertencentes ao seguinte conjunto : a,b,c,..z, A,B,C,...Z, 0,1,2,...,9, _

3. Não deve haver espaço em branco;
4. Não deve haver identificadores repetidos;
5. Existe distinção de maiúsculas e minúsculas;
6. Os nomes escolhidos devem ser claros a fim de explicitar seu conteúdo uso, mas
7. também não deve ser extenso para não dificultar a escrita.

2.3 Variáveis e Constantes

Unidades básicas de armazenamento das informações em programação.

As constantes referem-se a valores fixo que não podem ser alterados pelo programa. O modo como cada constante é representada depende do seu tipo.

Uma variável é um espaço de memória reservado para armazenar um certo tipo de dado e tendo um nome para referenciar o seu conteúdo. Uma variável pode conter, a cada instante, valores diferentes.

2.3.1 Declaração de Variáveis

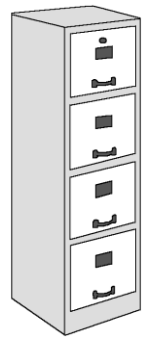
Precisamos declarar as variáveis para informar quais dados serão manipulados pelo programa. as informações variáveis são guardadas em dispositivos eletrônicos chamados “memória”.

A declaração de variáveis deve seguir a seguinte sintaxe:

```
tipo nome-da-variável1;  
tipo nome-da-variável2, nome-da-variável3;
```

Exemplo:

```
int idade,matricula;  
real media, total;  
char letra, nome[20];
```



2.3.2 Declaração de Constantes

Diferente das variáveis, as constantes mantêm seu valor ao longo do programa. Para indicar ao compilador que se trata de uma constante, a diretiva `#define` é utilizada.

Exemplo: `#define <identificador> <valor>`

Não é necessário colocar o ponto e vírgula no final e também não precisa informar o tipo de dado da constante. O valor de uma constante não pode ser modificado de maneira alguma.

O modo como cada constante é representada depende do seu tipo. Constantes caractere são colocadas entre apóstrofes, por exemplo, 'j'. Constantes cadeia de caractere (ou string) são colocadas entre aspas duplas, por exemplo, "Venus" e "jota". Constantes inteiro são especificadas como números sem componente fracional, por exemplo, 10 e 67. Constantes ponto flutuante requerem o uso de ponto decimal seguido pelo componente fracional do número, por exemplo, 10.78. Exemplos:

```
#define pi 3.1416
#define resposta 's'
#define disciplina "Matemática"
#define dias 7
```

2.4 Operações Básicas

Os operadores são o meio pelo qual incrementamos, decrementamos, comparamos e avaliamos dados no computador. Temos quatro tipos de operadores:

- Operador de Atribuição
- Operadores Aritméticos
- Operadores Relacionais
- Operadores Lógicos

2.4.1 Operador de Atribuição

Nome-da-Variavel = Valor, Variável ou Expressão;

Esse operador armazena um valor, o conteúdo de uma variável ou o resultado de uma expressão na variável. É importante que o tipo do Valor, Variável ou Expressão seja compatível com o tipo da variável.

Ex: MEDIA = 7;
MEDIA = X;
MEDIA = X + Y;

2.4.2 Operadores Aritméticos

Esses operadores serão utilizados em expressões para realizar operações aritméticas com variáveis.

Operador	Descrição
+	Adição
-	Subtração
*	Multiplicação
/	Divisão Inteira
%	Retorna o resto da divisão inteira de a por b

Exemplo: Suponha que $A = 6$, $B = 1$, $C = 16$ e $D = 6.5$, então:

$$A = A + 1 \Rightarrow A = 7$$

$$B = B * C \Rightarrow B = 16$$

$$C = C - A \Rightarrow C = 9$$

$$A = C \% A \Rightarrow A = 2 \text{ (resto da divisão)}$$

$$A = C/A \Rightarrow A = 4 \text{ (parte inteira da divisão)}$$

$$D = D/2 \Rightarrow A = 3.25 \text{ (divisão fracionária)}$$

Prioridade de Operadores

1º	Operações embutidas em parênteses “mais internos”
2º	Resto
3º	Multiplicação e/ou divisão
4º	Adição e/ou Subtração

Exemplos:

$$A = B + C * C \Rightarrow A = 1 + 16 * 16 \Rightarrow A = 1 + 256 \Rightarrow A = 257$$

$$A = (B + C) * C \Rightarrow A = (1 + 16) * 16 \Rightarrow A = 17 * 16 \Rightarrow A = 272$$

2.4.2.1 Biblioteca math.h

A biblioteca math.h possui funções úteis para a resolução de problemas que envolve questões matemáticas. A seguir serão apresentadas as principais funções desta biblioteca.

Funções trigonométricas

- double **sin** (double);
Exemplo: $y = \sin(x)$;
Retorna o seno de x
- double **cos** (double);
Exemplo: $y = \cos(x)$;
Retorna o cosseno de x
- double **tan** (double);
Exemplo: $y = \tan(x)$;
Retorna a tangente de x

Exponenciais e logaritmos

- double **exp** (double);
Exemplo: $y = \exp(x)$;
Retorna o número e elevado à potência x

- double **log** (double);
Exemplo: $y = \log(x)$;
Retorna o logaritmo de x na base e. Não use com x negativo.
- double **log10** (double);
Exemplo: $y = \log_{10}(x)$;
Retorna o logaritmo de x na base 10. Não use com x negativo.

Raiz e potência

- double **sqrt** (double);
Exemplo: $y = \sqrt{x}$;
Retorna a raiz quadrada de x. Não use com $x < 0$.
- double **pow** (double, double);
Exemplo: $p = \text{pow}(x, y)$;
Retorna x elevado à potência y.
Não use com $x = 0$ e $y < 0$. Não use com $x < 0$ e y não-inteiro. Caso especial: $\text{pow}(0, 0) = 1.0$.

Arredondamentos

- double **floor** (double);
Exemplo: $i = \text{floor}(x)$;
Retorna o maior inteiro que seja menor que ou igual a x, isto é, o único inteiro i que satisfaz $i \leq x < i+1$. Exemplo, se $x = 1.09$ será retornado 1.
- double **ceil** (double);
Exemplo: $j = \text{ceil}(x)$;
Retorna o menor inteiro que seja maior que ou igual a x, isto é, o único inteiro j que satisfaz $j-1 < x \leq j$. Exemplo, se $x = 1.09$ será retornado 2.

2.4.3 Operações Relacionais

São utilizados para comparar variáveis ou expressões, resultando num valor lógico (Verdadeiro ou Falso).

Operador	Descrição
==	Igual
!=	Diferente ($\neq$)
<	Menor
>	Maior
<=	Menor ou Igual ($\leq$)
>=	Maior ou Igual ($\geq$)

Alerta: O resultado de uma operação usando operadores relacionais só pode dar um dos valores: VERDADE ou FALSO, jamais dará outro valor.

Exemplo: Suponha que $A = 1$ e $B = 6$

$A > B \Rightarrow$ falso

$A \geq B \Rightarrow$ falso

$A < B \Rightarrow$ verdade

$A \leq B \Rightarrow$ verdade

$A == B \Rightarrow$ falso

2.4.4 Operadores Lógicos

São utilizados para avaliar expressões lógicas.

Uma expressão lógica representa a união de operações relacionais permitindo que o resultado de várias expressões relacionais sejam transformado em um único resultado lógico.

Operadores Lógicos	Operador em C
E	&&
OU	
NÃO	!

Esses operadores obedecem as seguintes tabelas:

CON1	CON2	OU
V	V	V
V	F	V
F	V	V
F	F	F

CON1	CON2	E
V	V	V
V	F	F
F	V	F
F	F	F

CON1	NÃO
V	F
F	V

Prioridade de Operadores

1º	Operações embutidas em parênteses “mais internos”;
2º	%
3º	Multiplicação e/ou divisão;
4º	Adição e/ou Subtração;
5º	Operadores Relacionais;
6º	Operadores Lógicos.

2.5 Atribuições reduzidas

C oferece muitos operadores de atribuição que são redução de outros operadores. Eles tomam a forma de $\text{op} =$, onde op pode ser $+$, $-$, $*$, $/$, $\%$. A expressão $f \text{op} = g$ é análoga a $f = f \text{op} g$. Por exemplo:

Expressão	É igual a
$a += 2$	$a = a + 2$
$s /= 7 + 2$	$s = s / (7 + 2)$

2.6 Operadores pré e pós-fixados

Os operadores pré e pós-fixados incrementam (++) ou decrementam (--) uma variável. Uma operação prefixada é realizada antes que o valor da variável seja utilizado. Uma operação pós-fixada é efetuada após a utilização da variável. Por exemplo, para uma variável *i* inteira com valor 5:

Expressão	Valor de <i>i</i> utilizado na avaliação	Valor da expressão	Valor final de <i>i</i>
5 + (i++)	5	10	6
5 + (i--)	5	10	4
5 + (++i)	6	11	6
5 + (--i)	4	9	4

2.7 Conversão de tipo (Casting)

Algumas vezes queremos, momentaneamente, modificar o tipo de dado representado por uma variável, isto é, queremos que o dado seja apresentado em um tipo diferente do qual a variável foi inicialmente declarada. Por exemplo: declaramos uma variável como `int` e queremos, momentaneamente, que seu conteúdo seja apresentado como `float`. Este procedimento é chamado de **conversão de tipo** ou *casting* (moldagem, em inglês).

Sintaxe: A sintaxe da instrução de conversão de tipo é: *(tipo) variável*

onde tipo é o nome do tipo ao qual queremos converter o dado armazenado em variável.

Exemplo: observe a conversão de tipo feita no exemplo abaixo:

```
int num;
float valor = 13.0;
num = (int)valor % 2;
```

No exemplo acima a variável `valor` foi declarada inicialmente como sendo do tipo `float` recebendo o valor inicial 13.0. Logo em seguida o conteúdo de `valor` é convertido para o tipo `int` para realizar a operação *resto* (%) com o inteiro 2. Aqui a conversão é necessária pois a operação *resto* somente pode ser feita com inteiros. É importante salientar que a conversão de tipo é feita com o **dado** armazenado em uma variável mas a **variável** continua tendo o seu tipo original. No exemplo acima a variável `valor` e os dados nela armazenados continuam sendo do tipo `float` após a conversão.

2.8 Comandos de Entrada e Saída de Dados

O objetivo de escrevermos programas é em última análise, a obtenção de resultados (Saídas) depois da elaboração de cálculos ou pesquisas (Processamento) através do fornecimento de um conjunto de dados ou informações conhecidas (Entradas).

2.8.1 Entrada de Dados

A execução da instrução de leitura pressupõe que os dados serão fornecidos do meio externo (teclado), e serão armazenados na memória (variáveis).

O comando `scanf` servirá para promover leitura de dados via teclado.

Forma Geral: **`scanf("string de controle", lista de argumentos);`**

string de controle:

- `%c` - leitura de caractere
- `%d` - leitura de números inteiros
- `%f` - leitura de números reais
- `%s` - leitura de caracteres

Lista de argumentos: variáveis onde os valores digitados via teclado deverão ser armazenados. A variável deve ser precedida do `&`.

Exemplo:

```
scanf("%d", &idade);
scanf("%f", &nota);
```

A lista de argumentos deve conter exatamente o mesmo número de argumentos quantos forem os códigos de formatação na `<string de controle>`. Se este não for o caso, diversos problemas poderão ocorrer - incluindo até mesmo o fechamento do programa.

Atenção: cadeias de caracteres devem ser lidas utilizando o comando `gets`. Não deve ser usado o `scanf`! Antes da leitura, deve ser utilizado o comando que limpa o *buffer* de entrada.

Exemplo:

```
char nome[20];
fflush(stdin); // obrigatório, limpa o buffer de entrada.
gets(nome); // usa gets e não scanf, não precisa de &.
```

2.8.2 Saída de Dados

A execução do comando de saída fará com que informações sejam mostradas na tela.

Forma Geral: **`printf("string de controle", lista de argumentos);`**

Códigos especiais:

<code>\n</code>	avanço de linha
<code>\t</code>	tabulação (tab)
<code>\"</code>	aspas duplas
<code>\\</code>	barra
<code>\a</code>	Alerta sonoro

Necessariamente você precisará ter tantos argumentos quantos forem os comandos de formatação na “string de controle”. Se isto não ocorrer, a tela exibirá sujeira ou não exibirá qualquer dado.

Códigos de impressão formatada:

<code>%c</code>	caractere
<code>%d</code>	inteiro
<code>%f</code>	real
<code>%s</code>	string
<code>%%</code>	o caractere '%'

Exemplos:

```
printf("Uma linha");  
  
printf("Uma linha\nDuas linhas");  
  
printf("Os números são: %d e %d\n", 7, 8);  
  
printf("Caruaru está a %d Km de Recife", dist);  
  
printf("%d%% de %d = %f\n", p, x, x*(p/100.0));
```

Formatação

O tamanho de campos de impressão é indicado logo após o '%' e antes do tipo do campo:

```
printf("A quantidade de alunos eh %4d!\n", 44);  
  
printf("A quantidade de alunos eh %-4d!\n", 44);  
  
printf("R$ %.2f!\n", 1234.5632);  
  
printf("R$ %10.2f!\n", 1234.5632);  
  
printf("R$ %-10.2f!\n", 1234.5632);
```

2.9 Palavras Reservadas do C

A linguagem C, como todas as outras linguagens de programação, consiste de palavras reservadas e regras de sintaxe que se aplicam a cada palavra reservada. Uma palavra reservada é essencialmente um

comando e, na maioria das vezes, as palavras reservadas de uma linguagem definem o que pode ser feito e como pode ser feito.

Na linguagem C são 32 palavras reservadas. Todas as palavras reservadas do C são minúsculas. Uma palavra reservada não pode ser usada para nenhum outro propósito em um programa. A tabela abaixo mostra as palavras reservadas conforme definido pelo padrão ANSI para a linguagem C.

auto	default	float	unsigned
break	do	for	return
case	double	goto	short
char	else	union	signed
const	typedef	if	sizeof
switch	while	int	static
volatile	enum	long	struct
continue	extern	register	void

2.10 Estrutura de um programa básico em C

- Inclusão de bibliotecas (se necessário)
- Inclusão das constantes
- Comentários (se desejar)
- Programa principal
 - Declaração de variáveis
 - Sequência de comandos

2.10.1 Comentários

Os comentários são utilizados para documentar um programa, bem como podem ser úteis para encontrar erros no código.

Comentários podem ser colocados em qualquer lugar de seu programa. Comentários começam com barra-asterisco (/*) e terminam por asterisco-barra (*). O compilador irá ignorar tudo o que encontrar entre estes dois símbolos.

Inserir comentários em seus programas é fundamental para que se tenha uma boa documentação. A documentação auxilia no entendimento do programa.

Asteriscos dentro de comentários podem ser colocados livremente. A seguir o exemplo de um cabeçalho que pode ser colocado para facilitar a documentação dos programas.

```

/*****
* Programa: corrida.c
* Programador: José Alves
* Data: 12/01/2007
* Programa para imprimir o resultado de uma corrida
*****/

```

Importante: Não são permitidos os símbolos `/*` ou `*/` no interior de um comentário:

```
/* Estou escrevendo /* um comentário ilegal */
```

A maioria dos compiladores C também aceita comentários de apenas uma linha, ou seja, que terminam no final da linha. Neste caso, o comentário é feito colocando-se `//` no início do texto de comentário. Neste caso não tem um símbolo para fechar o comentário.

```
#include <stdio.h>
#include <conio.h>
// programa para calcular a media
main()
{ float m; // variável que armazena a media
```

Exemplos:

```
#include <stdio.h>
#include <conio.h>
/* programa para escrever frase na tela */
main()
{ printf(" ALO RECIFE!");
  getche();
}
```

```
#include <stdio.h>
#include <conio.h>
// programa para calcular o quadrado de um numero qualquer
main()
{ int n, q;
  printf("Quadrado de um Numero\n");
  printf("Numero: ");
  scanf("%d",&n);
  q = n * n;
  printf("\n\ao quadrado de %d = %d",n,q);
  getche();
}
```

```
#include <stdio.h>
#include <conio.h>
#include <math.h>
//calculo da raiz quadrada de um numero inteiro qualquer
main()
{ int num;
  float r;
  printf("\nRaiz quadrada de um numero\n\n");
  printf("Digite o numero: ");
  scanf("%d",&num);
  r = sqrt(num); // calculo da raiz quadrada
  printf("\nRaiz quadrada de %d = %.2f",num,r);
  getche();
}
```

Alguns termos comuns

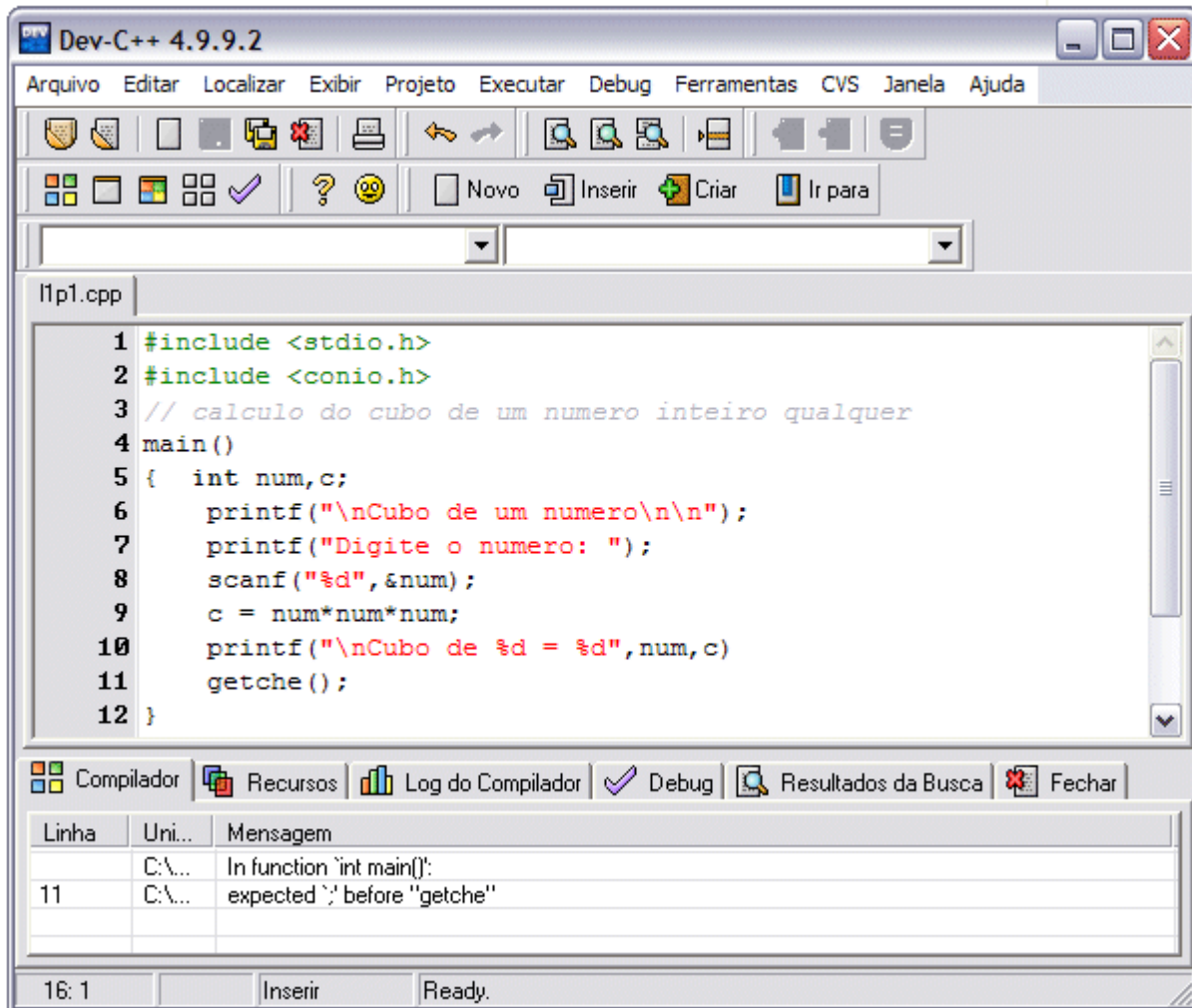
Tempo de compilação: eventos que ocorrem enquanto seu programa esta sendo compilado.

Tempo de execução: eventos que ocorrem quando seu programa esta sendo executado.

Biblioteca: é um arquivo contendo as funções padrão que seu programa poderá usar.

Código fonte: é o texto de um programa que o usuário pode ler,comumente chamado de programa.

2.11 Conhecendo o Dev-C++



Exercícios Propostos I



1. Identifique os tipos abaixo:

- | | | | |
|----------|--------|-----------|------------|
| 1) 1000 | 2) "0" | 3) - 1.56 | 4) VERDADE |
| 5) - 456 | 6) 34 | 7) "C" | 8) 45.8976 |

2. Quais os nomes válidos de identificadores?

- | | | | |
|-------------|-----------------|--------------|-----------------|
| 1) ENDERECO | 2) 21BRASIL | 3) FONE\$COM | 4) NOME_USUARIO |
| 5) USUARIO6 | 6) NOME*USUARIO | 7) END*A-6 | 8) #MEDIA |

3. Com as declarações abaixo:

int NUM, X, SOMA;

float MEDIA, K, L;

Assinalar os comandos de atribuição inválidos:

- | | |
|---------------------|--------------------------|
| () SOMA = NUM + 2; | () L = SOMA % K; |
| () MEDIA = SOMA | () SOMA + 2 = NUM + 10; |
| () X = X + 1; | () NUM = SOMA; |
| () NUM = K + L | () X = SOMA/NUM; |

4. Dados os valores inteiros abaixo:

MEDIA	NOTA	TOTAL	Z	K	X	I	J	A
0	2	10	0	7	4	80	-1	-4

Quais os valores resultantes depois da execução dos passos a seguir:

01) X = TOTAL/NOTA;	17) J = A - 15;
02) X = X - X + 1;	18) TOTAL = MEDIA + A + NOTA;
03) X = X+1;	19) A = MEDIA;
04) X = X+8;	20) MEDIA = NOTA;
05) X = X-2;	21) NOTA = A;
06) X = X+1;	22) I = 20;
07) X = X+J;	23) K = I % X;
08) NOTA = X;	24) Z = J / 5;
09) TOTAL = NOTA + X + TOTAL;	25) K = I % 5;
10) MEDIA = J + K;	26) MEDIA = NOTA / A;
11) A = A + K + 4;	27) TOTAL = I + A;
12) MEDIA = 3;	28) I = 0;
13) I = 8 / 3;	29) I = I+2*A;
14) I = A % MEDIA;	30) I = I+X;
15) Z = A / MEDIA;	31) I = I*2;
16) K = 2 + X * X -2;	32) J = 3 * 4 + 27/I;

5. Quais os valores resultantes de cada linha do quadro abaixo:

A	B	C	D	$A+B*C>D$	$(D+C)*(A+B) \leq 35$
2	2	3	9		
4	3	2	2		
1	7	1	3		
3	3	3	4		
5	1	5	20		
2	7	2	18		
7	6	4	2		

6. De acordo com os valores das variáveis abaixo, responda as expressões com VERDADE ou FALSO:

A	B	C	D	E
10	3	2	2	25

- a) $(A == D)$
- b) $((A > B) \&\& (C == D))$
- c) $((E + 2 < B + C) \&\& (A == 10))$
- d) $((D * E > 100) || (B < 5))$
- e) $((A - 9 > 0) || (D - C == 0))$
- f) $((E == 25) \&\& (A > B + C + D) \&\& (C < B))$
- g) $((B > C) || (A != 5) || (C < E + A))$
- h) $((E != 25) \&\& (D == 2) || (C > 0))$

7. Faça um programa que leia um número qualquer e imprima o seu cubo.

8. Faça um programa que leia um número qualquer e imprima a sua raiz quadrada.

9. Faça um programa que leia um valor em graus Fahrenheit e imprima o seu equivalente em graus Celsius.

$$C = (F - 32) * 5 / 9$$

10. Faça um programa para ler um valor em graus Celsius e imprime o valor correspondente em graus Fahrenheit.

11. Faça um programa para calcular o volume de uma esfera de raio r, onde r é o valor lido.

$$V = \frac{4\pi r^3}{3}$$

12. Faça um programa que leia a velocidade de um automóvel em km/h e imprima em m/s. Nota: 1 m/s equivale a 3,6 km/h.

13. Faça um programa para ler o raio de um círculo, calcular e escrever a sua área.

14. A casa de câmbio Money trabalha com trocas de real para dólares. Contudo, as trocas em dólares são feitas usando somente notas de US\$10 e moedas de 50 cents. Faça um programa que leia um certo valor em reais e a cotação do dólar no dia e calcule o número de notas de US\$10 e moedas de 50¢ que o usuário vai receber em troca dos reais. Também deve ser informada a quantia em reais que não será convertida.

15. Faça um programa para ler o salário mensal e o percentual de reajuste. Calcular e escrever o valor do novo salário.

16. Faça um programa para ler o número de votos brancos, nulos e válidos. Calcular e escrever o percentual que cada um representa em relação ao total de eleitores.

17. Suponha que você foi ao supermercado e comprou:

N latas de Nescau com custo unitário Q

L litros de leite com custo unitário P

B quilos de banana com custo unitário T

Faça um programa que imprima: total gasto com cada produto e o total gasto no mercado.

18. Uma sorveteria produz 3 tipos de picolés. O preço de cada um deles é: 0.5, 0.6 e 0.1, respectivamente. Sendo informado ao programa a quantidade vendida do tipo 1, do tipo 2 e do tipo 3, gere o seguinte relatório:

a) o total arrecadado com os picolés do tipo 1, 2, 3.

b) o total arrecadado pela sorveteria.

19. Faça um programa que leia o número total de prestações de um consórcio, o total de prestações pagas e o valor atual da prestação. O programa deve calcular e apresentar o saldo devedor atual e a quantidade de prestações sem pagar.

20. Faça um programa que leia a matrícula do vendedor, seu salário fixo, o valor total de vendas efetuada por ele e o percentual que ganha sobre as vendas. Escreva o salário total do vendedor.

21. Faça um programa que lê a matrícula de um funcionário, número de horas trabalhadas, valor da hora trabalhada, número de filhos com idade menor que 14 anos e calcule o salário deste funcionário. Para cada filho menor, ele tem um acréscimo de R\$30,00 no salário.

22. O governo acaba de liberar R\$10.000.000 para construção de casas populares. Cada casa custa o equivalente a 150 salários mínimos. Faça um programa que leia o valor do salário mínimo e calcule a quantidade de casas possíveis de se construir.

23. Faça um programa que leia um valor em Real (R\$) e calcule o menor número possível de notas de 50,00, 10,00, 5,00 e 1,00 real em que o valor lido pode ser decomposto. Escrever o valor lido e a relação de notas necessárias. Suponha que o sistema monetário não utiliza centavos.

Exemplo: Valor: R\$ 347,00

Notas de R\$ 50,00: 6

Notas de R\$ 10,00: 4

Notas de R\$ 5,00: 1

Notas de R\$ 1,00: 2

24. Dada as seguintes entradas: km inicial, km final, litros consumidos, preço do litro de combustível. Faça um programa que imprima o seguinte Relatório: distancia percorrida, Valor total gasto e Km por litro.

25. A turma C é composta de 60 alunos, e a turma D de 20 alunos. Faça um programa que leia o percentual de alunos reprovados na turma C, o percentual de aprovados na turma D, calcule e imprima:

- a) O número de alunos reprovados na turma C.
- b) O número de alunos reprovados na turma D.

26. Um motorista de táxi deseja calcular o rendimento de seu carro na praça. Sabendo-se que o preço do combustível é de R\$2.20, faça um programa para ler: a marcação do odômetro (Km) no início do dia, a marcação (Km) no final do dia, a quantidade de litros de combustível gasto e o valor total (R\$) recebido dos passageiros. Calcular e escrever: a média do consumo em km/l e o lucro (líquido) do dia.

27. Uma loja vende bicicletas com um acréscimo de 50 % sobre o seu preço de custo. Ela paga a cada vendedor 2 salários mínimos mensais. Faça um programa que leia o valor do salário mínimo, o preço de custo de uma bicicleta, o número de bicicletas vendidas e a quantidade de funcionários. Calcule e imprima o lucro (líquido) da loja.

28. A equipe Ferrari deseja calcular o número mínimo de litros que deverá colocar no tanque de seu carro para que ele possa percorrer um determinado número de voltas até o primeiro reabastecimento. Faça um programa que leia o comprimento da pista (em km), a quantidade de voltas a serem percorridas no grande prêmio, o número de reabastecimentos desejados, e o consumo de combustível do carro (em km/l). Calcular e escrever o número mínimo de litros necessários para percorrer até o primeiro reabastecimento.

29. Faça um programa que leia os preços de uma mercadoria em abril e maio. Tendo como base a variação dos preços desta mercadoria, calcule a taxa de inflação.

30. Faça um programa que leia um valor inteiro positivo e menor que 1000, armazene numa variável inteira e imprima a soma dos dígitos que formam o valor.

Exemplo: 397 => soma=19

31. O custo ao consumidor de um carro novo é a soma do custo de fábrica com a percentagem do distribuidor e dos impostos (aplicados ao custo de fábrica). Sabe-se que a percentagem do distribuidor é de 28% e os impostos de 45%, faça um programa que leia o custo de fábrica de um carro e escreva o custo ao consumidor.

32. Dados os comprimentos dos catetos de um triângulo retângulo nas variáveis LADO1 e LADO2, faça um programa que calcule o valor da HIPOTENUSA.

33. Faça um programa que leia o tempo de duração de um evento em uma fábrica expressa em segundos e mostre-o expresso em horas, minutos e segundos.

34. Faça um programa que, tendo como dados de entrada dois pontos quaisquer no plano, P(x1,y1) e P(x2,y2), escreva a distância entre eles. A fórmula que efetua tal cálculo é:

$$d = \sqrt{\left((x_2 - x_1)^2 + (y_2 - y_1)^2 \right)}$$

35. Faça um programa que leia dois valores, A e B, e os imprima ao contrário, isto é, se em A for lido 7, e em B 11, quando você for escrever A, será impresso 11 e quando escrever B será impresso 7. Neste algoritmo você só pode usar as variáveis A e B. Dica: operadores aritméticos resolvem este problema.

36. Uma empresa de ferragens produz 3 tipos de peças mecânicas: parafusos, porcas e arruelas. Sabe-se que é dado um desconto de: 10% por porca vendida, 20% por arruela vendida, 30% por parafuso vendido.

Dados de entrada: preço unitário da porca, da arruela e do parafuso; quantidade de porcas, arruelas e parafusos solicitados pelo cliente.

Dados de saída: total pago pelas porcas, arruelas e parafusos; total de desconto dado ao cliente; total a ser pago pelo cliente.

37. Uma revendedora de carros usados paga a seus funcionários vendedores, um salário fixo por mês, mais uma comissão também fixa para cada carro vendido e mais 5% do valor das vendas por ele efetuadas. Escrever um programa que lê o número do vendedor, o número de carros por ele vendidos, o valor de cada carro, o salário fixo e o valor que recebe por carro vendido e calcula o salário a ser pago a este vendedor, escrevendo o número do vendedor e seu salário total.

38. Um hotel com 40 apartamentos resolveu fazer promoções para os fins de semana fora da alta temporada, isto é, nos meses de abril, maio, junho, agosto, setembro outubro e novembro. A taxa da promoção é de 22% da diária normal. A expectativa é aumentar a taxa de ocupação de 40 para 70%. Supondo que as expectativas se confirmem, escrever um programa que lê a diária normal e calcula e escreve as seguintes informações:

- a) O valor da diária no período da promoção.
- b) O valor médio arrecadado sem a promoção, durante um mês.
- c) O valor médio arrecadado com a promoção, durante um mês.
- d) O lucro ou prejuízo mensal com a promoção.

Capítulo 3 - Comandos de Seleção

3.1 Introdução

Usado para tomar decisões, ou seja desviar a execução do programa de acordo com uma condição, podendo ser simples ou composta.

O comando de seleção é utilizado quando há a necessidade de avaliar certas possibilidades dos valores de variáveis e, de acordo com o resultado dessa análise, executar um conjunto específico de comandos.

3.2 IF...ELSE (SE...SENÃO)

A estrutura if...else é a mais simples estrutura de seleção do C. Esta estrutura permite executar um entre vários blocos de instruções. O controle de qual bloco será executado será dado por uma *condição* (expressão lógica ou numérica). Esta estrutura pode se apresentar de modos ligeiramente diferentes. Nesta seção vamos apresentar separadamente cada uma das possibilidades de sintaxe.

Se Simples

Primeiro Caso: if com um único comando

if (Condição) Comando;	if (x==1) a = a + 3;
---------------------------	-------------------------

Segundo Caso: if com mais de um comando

if (Condição) { Comando; ... Comando; }	if (x==1) { a = a + 3; z = z * 2; h = 12/6; }
--	--

Se Composto**Primeiro Caso:** if e else com um único comando

<pre>if (Condição) Comando; else Comando;</pre>	<pre>if (media >= 7) printf("\nAprovado"); else printf("\nReprovado");</pre>
---	---

Segundo Caso: if com mais de um comando e else com um comando

<pre>if (Condição) { Comando; ... Comando; } else Comando;</pre>	<pre>if (sal < 260) { bonus = sal * 0.1; nsal = sal + bônus; } else printf("\nsem bonus");</pre>
--	---

Terceiro Caso: if com um comando e else com mais de um comando

<pre>if (Condição) Comando; else { Comando; ... Comando; }</pre>	<pre>if (sal >= 260) printf("\nsem bonus"); else { bonus = sal * 0.1; nsal = sal + bônus; }</pre>
--	--

Quarto Caso: if e else com mais de um comando

<pre>if (Condição) { Comando; ... Comando; } else { Comando; ... Comando; }</pre>	<pre>if (sal >= 260) { bonus = sal * 0.2; nsal = sal + bônus; } else { bonus = sal * 0.4; sf = num_filho * 20; nsal = sal + bonus + sf; }</pre>
---	--

OBS: A condição sempre deve vir entre parênteses.

Exemplo: Verificar se o aluno foi aprovado ou reprovado.

```
#include <stdio.h>
#include <conio.h>
main()
{ float med;
  printf("Caderneta de Notas\n\n");
  printf("Media do Aluno: ");
  scanf("%f",&med);
  if (med>=7)
    printf("\n\nAluno aprovado com media %.1f\n",med);
  else
    printf("\n\nAluno reprovado com media %.1f\n",med);
  getche();
}
```

Se Aninhado(Se dentro de outro Se)

```
#include <stdio.h>
#include <conio.h>
main()
{ int a, b;
  printf("Compara Numeros\n\n");
  printf("A: ");
  scanf("%d",&a);
  printf("B: ");
  scanf("%d",&b);
  if (a > b)
    printf("\n\n%d eh maior que %d\n",a,b);
  else
  { if (a < b)
    printf("\n\n%d eh menor que %d\n",a,b);
    else
      printf("\n\nA e B são iguais: %d\n",a);
  }
  getche();
}
```

3.3 SWITCH (Escolha)

A estrutura switch...case é uma estrutura de decisão que permite a execução de um conjunto de instruções a partir pontos diferentes conforme o resultado de uma expressão inteira de controle. O resultado desta expressão é comparado ao valor de cada um dos rótulos, e as instruções são executadas a partir desde rótulo.

Escolha simples

<pre>switch (variavel) { case valor : Comando; ...break; ... case valor: Comandos; ... break; }</pre>	<pre>switch (cargo) { case 1: sal = sal + 200; break; case 2: sal = sal + 400; break; case 3: sal = sal + 600; break; }</pre>
---	---

Observe que “break” serve para terminar a sequência de comandos em execução, por serem opcionais, se forem suprimidos permitem que o “case” a seguir seja executado, sem haver qualquer quebra na sequência do processamento.

Escolha Composto

<pre>switch (variavel) { case valor : Comando; ...break; ... case valor: Comandos; ... break; default: comandos; ... break; }</pre>	<pre>switch (cargo) { case 1: sal = sal + 200; break; case 2: sal = sal + 400; break; case 3: sal = sal + 600; break; default : sal = sal + 100; break; }</pre>
---	---

Exemplo:

```
#include <stdio.h>
#include <conio.h>
main()
{ int cod, quant;
  printf("Supermercado Boa Compra\n\n");
  printf("Codigo do Produto: ");
  scanf("%d",&cod);
  printf("Quantidade: ");
  scanf("%d",&quant);
  switch (cod)
  {
    case 102: printf("\n\nTotal a pagar: R$%.2f\n",quant*5.3); break;
    case 103: printf("\n\nTotal a pagar: R$%.2f\n",quant*6.0); break;
    case 104: printf("\n\nTotal a pagar: R$%.2f\n",quant*3.2); break;
    case 105: printf("\n\nTotal a pagar: R$%.2f\n",quant*2.5); break;
    default: printf("\n\nCodigo de produto Invalido!\n"); break;
  }
  getch();
}
```

Escolha com Letras

```
printf("\n\nSair (s-sim/n-nao)? ");
fflush(stdin);
scanf("%c",&letra);
switch (letra)
{
    case 's': case 'S': printf("voce irá sair do programa"); break;
    case 'n': case 'N': printf("voce irá repetir mais uma vez"); break;
    default: printf("escolha invalida"); break;
}
```



Exercícios Propostos II

1. Calcule a média aritmética das 3 notas de um aluno e mostre, além do valor da média, uma mensagem de "Aprovado", caso a média seja igual ou superior a 7 ou a mensagem "Reprovado", caso a média seja menor que 7.
2. Elaborar um programa que lê 2 valores A e B (inteiros) e informa se A é divisível por B ou não.
3. Faça um programa que leia um número inteiro e mostre uma mensagem indicando se este número é par ou ímpar, e se é positivo ou negativo.
4. As maçãs custam R\$ 0,30 cada uma. Se forem compradas menos do que uma dúzia, e R\$ 0,25 se forem compradas pelo menos doze. Faça um programa que leia o número de maçãs compradas, calcule e escreva o valor total da compra.
5. Faça um programa que verifique a validade de uma senha fornecida pelo usuário. A senha válida é o número 12345. Devem ser impressas as seguintes mensagens:
 - ACESSO PERMITIDO caso a senha seja válida.
 - ACESSO NEGADO caso a senha seja inválida.
6. Tendo como dados de entrada a altura e o sexo(1-feminino/2-masculino) de uma pessoa, construa um programa que calcula e escreve seu peso ideal, utilizando as seguintes fórmulas:
 - para homens: $(72.7 * \text{altura}) - 58$
 - para mulheres: $(62.1 * \text{altura}) - 44.7$
7. Faça um programa que leia o número da conta bancária, o saldo, o tipo de operação (1-depósito/2-retirada) e o valor da operação. O algoritmo deve calcular e mostrar o novo saldo. Mostrar uma mensagem caso o saldo fique negativo.
8. Elabore um programa que dada a idade de um nadador classifica-o em uma das seguintes categorias:
 - infantil A = 5 - 7 anos
 - infantil B = 8-10 anos
 - juvenil A = 11-13 anos
 - juvenil B = 14-17 anos
 - adulto = maiores de 18 anos
9. Considere definidas as seguintes faixas sobre os números inteiros:
 - Faixa1 - até 100
 - Faixa2 - de 101 até 200
 - Faixa3 - de 201 até 300
 - Faixa4 - de 301 até 400
 - Faixa5 - de 401 até 500
 - Faixa6 - acima de 500

Faça um programa que leia um inteiro N e imprima a faixa à qual ele pertence.

10. Um hotel cobra R\$ 50,00 reais a diária e mais uma taxa de serviços. A taxa de serviços é de:

2,50 por dia, se número de diárias <15

2,00 por dia, se número de diárias =15

1,50 por dia, se número de diárias >15

Faça um programa que ler a quantidade de dias que o hospede ficou no hotel e imprime a taxa e total a pagar.

11. Elaborar um programa que irá ler três números inteiros diferentes e informa qual é o maior valor.

12. Elaborar um programa que irá ler quatro inteiros diferentes e informa qual é o menor valor.

13. Um banco concederá um crédito especial aos seus clientes, variável com o saldo médio no último ano. Faça um programa que leia o saldo médio de um cliente e calcule o valor do crédito de acordo com a tabela abaixo. Mostre uma mensagem informando o saldo médio e o valor do crédito.

Saldo médio	Percentual
De 0 a 200	nenhum crédito
De 201 a 400	20% do valor do saldo médio
De 401 a 600	30% do valor do saldo médio
Acima de 601	40% do valor do saldo médio

14. Um vendedor precisa de um programa que calcule o preço total devido por um cliente. O algoritmo deve ler o código de um produto e a quantidade comprada. Calcular o preço total, usando a tabela abaixo. Mostre uma mensagem no caso de código inválido.

Código	Preço unitário
102	R\$ 5,30
103	R\$ 6,00
104	R\$ 3,20
105	R\$ 2,50

15. Uma empresa concederá um aumento de salário aos seus funcionários, variável de acordo com o cargo, conforme a tabela abaixo. Faça um programa que leia o salário e o cargo de um funcionário(1-Gerente,2-Engenheiro,3-Técnico,4-Auxiliar) e calcule o novo salário. Se o cargo do funcionário for inválido deverá imprimir uma mensagem de erro. Como resultado mostre o salário antigo, o novo salário e a diferença.

Cargo	Percentual
Gerente	10%
Engenheiro	20%
Técnico	30%
Auxiliar	40%

16. Escrever um programa que lê a matrícula do aluno e suas 3 notas. Calcular a média e verificar qual o seu conceito, conforme a tabela:

Média	Conceito
$\geq 9,0$	A
$\geq 7,5$ e $< 9,0$	B
$\geq 6,0$ e $< 7,5$	C
$\geq 4,0$ e $< 6,0$	D
$< 4,0$	E

O programa deve escrever a matrícula do aluno, suas notas, a média, o conceito correspondente e a mensagem: APROVADO se o conceito for A,B ou C e REPROVADO se o conceito for D ou E.

17. São dados um número n e quatro outros a, b, c, d , correspondentes, respectivamente, às extremidades dos intervalos $[a,b]$ e $[c,d]$, sendo $a < b, c < d$ e $a < c$. Determinar se n pertence somente ao intervalo $[a,b]$ ou somente ao intervalo $[c,d]$ ou, se n pertence a ambos ou se n não pertence a nenhum dos dois. Em cada caso escrever uma mensagem conveniente.

18. Escrever um programa que lê um conjunto de 4 valores I, A, B, C , onde I é um valor inteiro e positivo e A, B, C , são quaisquer valores reais e os escreva. A seguir:

- a) Se $I = 1$ escrever os três valores A, B, C em ordem crescente.
- b) Se $I = 2$ escrever os três valores A, B, C em ordem decrescente.

19. Escrever um programa que lê o número de um funcionário, o número de horas trabalhadas, o valor que recebe por hora, o número de filhos com idade inferior a 14 anos, o tempo de serviço do funcionário e o valor do salário família por filho. Calcular: salário bruto, o desconto do INSS (8.5% do salário bruto), salário família, IR, Adicional.

o IR (imposto de renda) é calculado como segue:

- Se salário bruto > 1500 então IR = 15% do salário bruto
- Se salário bruto > 500 e salário bruto ≤ 1500 então IR = 8% do salário bruto
- Se salário bruto ≤ 500 então IR = 0

o adicional é calculado conforme o especificado:

- Se tempo de serviço for até 5 anos o adicional é igual a 2% do salário bruto.
- Se tempo de serviço estiver entre 6 e 15 anos o adicional é igual a 3.5% do salário bruto.
- Se tempo de serviço for maior que 15 anos então adicional é igual a 5.5% do salário bruto.

Calcular o salário líquido do funcionário. Mostrar o número do funcionário, salário bruto, total dos descontos, adicional e salário líquido.

20. Faça um programa para ler a base e a altura de um retângulo, a base e a altura de um triângulo e o raio de uma circunferência e mostrar qual a maior área.

21. Escrever um programa que lê a hora de início e hora de término de um jogo, ambas subdivididas em dois valores distintos : horas e minutos. Calcular e escrever a duração do jogo, também em horas e minutos, considerando que o tempo máximo de duração de um jogo é de 24 horas e que o jogo pode iniciar em um dia e terminar no dia seguinte.

22. Um determinado posto rodoviário faz a apreensão de caminhões com carga superior ao permitido fazendo a pesagem dos caminhões que passam por uma rodovia. O posto obtém o peso da carga do caminhão a partir da seguinte formula: $\text{Peso Carga} = \text{Peso Final} - \text{Peso inicial}$ onde, Peso inicial é o peso do caminhão antes de ser colocada a carga em Toneladas, Quilos e Gramas. Peso Final é o peso do caminhão após a carga ser colocada em toneladas, quilos e gramas. Assim, fazer um programa que leia para um caminhão: o peso inicial (toneladas, quilos e gramas), o peso final (toneladas, quilos e gramas) e a carga máxima permitida. Se a carga for superior ao permitido escrever a mensagem: "Carga superior ao permitido" e a quantidade que passou do permitido. Caso contrário escrever "Carga OK!".

Exemplo: Peso Inicial: 1 ton. 10 kg. e 100 g.

Peso Final : 2 ton. 9 kg. e 400 g.

Carga Maxima Permitida: 700Kg

Peso Carga: 999 Kg. 300 g.

Carga superior ao permitido. Passaram 199 Kg e 300 g.

23. Fazer um programa que leia uma data (dia e mês) e informe se é primavera, verão, outono ou inverno. Sabe-se que as estações, no hemisfério sul, começam nas seguintes datas:

- Primavera: 22/Set
- Verão: 21/Dez
- Outono: 20/Mar
- Inverno: 20/Jun

24. Faça um programa para ler o peso de uma encomenda, tipo de entrega(1-sedex, 2-sedex 10) e região onde será entregue a encomenda(1- Norte, 2-Nordeste, 3-Centro-Oeste, 4-Sudeste, 5-Sul). Calcular o valor de postagem baseado nas tabelas abaixo:

Até 1 Kg	R\$ 5,00
De 1 Kg a 5 Kg	R\$ 10,00
Acima de 5 Kg	R\$ 15,00 + R\$ 3,00 por Kg excedente

Para entrega por Sedex	Adicionar R\$ 9,00
Para entrega por Sedex 10	Adicionar R\$ 11,00

Para entregar na região Norte	Adicionar R\$ 3,00
Para entregar na região Nordeste	Adicionar R\$ 2,00
Para entregar na região Centro-Oeste	Adicionar R\$ 6,00
Para entregar na região Sudeste	Adicionar R\$ 5,00
Para entregar na região Sul	Adicionar R\$ 7,00

Capítulo 4 - Comandos de Repetição

4.1 Introdução

A estrutura de repetição permite que um bloco de instruções seja executado repetidamente uma quantidade controlada de vezes.

4.2 FOR (PARA)

A estrutura costuma ser utilizada quando se quer um número determinado de repetições. A contagem dos ciclos é feita por uma variável chamada de **contador**. A estrutura for também é chamada de estrutura de **repetição com contador**. Sua sintaxe é a seguinte:

<pre>for (var-controle = valor-inicial; condição; incremento) comando;</pre>
<pre>for (var-controle = valor-inicial; condição; incremento) { comando; comando; }</pre>
<pre>for (var-controle = valor-inicial; condição; decremento) comando;</pre>

Exemplos:

<pre>#include <conio.h> #include <stdio.h> main () { int i; for (i=1; i<=10; i++) printf("\n %d", i); getch(); }</pre>	<pre>#include <conio.h> #include <stdio.h> main () { int i; for (i=10; i>=1; i--) printf("\n %d", i); getch(); }</pre>
---	---

```
#include <stdio.h>
#include <conio.h>
// Calcula Bônus do Cliente
main()
{ float valor;
  int cod, i;
  printf("Loja Compre Tudo\n\n");
  for (i=1; i<=5; i++)
  { printf("Codigo do Cliente %d: ",i);
    scanf("%d",&cod);
    printf("Valor das compras: ");
    scanf("%f",&valor);
    if (valor<5000)
      printf("\n\nValor do bonus R$ %.2f\n", valor*0.1);
    else
      printf("\n\nValor do bonus R$ %.2f\n", valor*0.15);
    printf("tecle enter para continuar...\n\n");
    getche();
  }
}
```

```
#include <conio.h>
#include <stdio.h>
//este programa escreve os quadrados dos números de 1 a 20
main ()
{ int i;
  for (i=1; i<=20; i++)
    printf("Quadrado de %d = %d\n",i, i*i);
  getche();
}
```

4.3 WHILE (ENQUANTO – teste no início)

Essa estrutura significa que enquanto a condição for verdadeira, o comando será executado repetidamente. Se a condição for falsa, então a repetição será interrompida. O while avalia a condição antes da primeira interação, isto significa que, eventualmente, pode não ocorrer sequer a primeira interação.

A sintaxe do While é:

while (condição) comando;	while (condição) { comando; ...; comando; }
------------------------------	---

Exemplo:

```
#include <stdio.h>
#include <conio.h>
// Calcula Bônus do Cliente
main()
{
    float valor;
    int cod, cont;
    printf("Loja Compre Tudo\n\n");
    cont = 1;
    while (cont<=5)
    { printf("Codigo do Cliente %d: ",cont);
      scanf("%d",&cod);
      printf("Valor das compras: ");
      scanf("%f",&valor);
      if (valor<5000)
          printf("\n\nValor do bonus R$ %.2f\n", valor*0.1);
      else
          printf("\n\nValor do bonus R$ %.2f\n", valor*0.15);
      printf("tecle enter para continuar...\n\n");
      getch();
      cont++;
    }
}
```

```
#include <conio.h>
#include <stdio.h>
//este programa escreve os quadrados dos números de 1 a 20
main ()
{ int cont;
  cont = 1;
  while (cont <=20)
  { printf("Quadrado de %d = %d\n",cont, cont*cont);
    cont++;
  }
  getch();
}
```

4.4 DO...WHILE (Faça...enquanto - teste no final)

Repete um bloco de instruções enquanto a condição é verdadeira. A diferença é que a condição só é testada no final. Sua sintaxe é:

do comando; while (condição);	do { comando; ...; comando; } while (condição);
-------------------------------------	---

Exemplos:

```
#include <stdio.h>
#include <conio.h>
// Calcula Bônus do Cliente
main()
{
    float valor;
    int cod, cont;
    printf("Loja Compre Tudo\n\n");
    cont = 1;
    do
    {
        printf("Codigo do Cliente %d: ", cont);
        scanf("%d", &cod);
        printf("Valor das compras: ");
        scanf("%f", &valor);
        if (valor < 5000)
            printf("\n\nValor do bonus R$ %.2f\n", valor * 0.1);
        else
            printf("\n\nValor do bonus R$ %.2f\n", valor * 0.15);
        printf("tecle enter para continuar...\n\n");
        getch();
        cont++;
    } while (cont <= 5);
}
```

```
#include <stdio.h>
#include <conio.h>
// Calcula Bônus do Cliente
main()
{
    float valor;
    int cod, op;
    printf("Loja Compre Tudo\n\n");
    do
    {
        printf("Codigo do Cliente: ");
        scanf("%d", &cod);
        printf("Valor das compras: ");
        scanf("%f", &valor);
        if (valor < 5000)
            printf("\n\nValor do bonus R$ %.2f\n", valor * 0.1);
        else
            printf("\n\nValor do bonus R$ %.2f\n", valor * 0.15);
        printf("\n\nDeseja continuar(1-sim/2-não)? :");
        scanf("%d", &op);
    } while (op == 1);
}
```

4.5 Comandos Desestruturadores

Break

Vimos anteriormente o comando “break” finalizando opções do comando “switch”. Este comando pode ser usado para forçar o encerramento de uma repetição. Sintaxe do break: `break;`

Continue

O comando “continue” funciona de maneira análoga ao “break”, contudo ao invés de forçar o encerramento da repetição, força nova iteração saltando o código entre seu uso e a marca de término da repetição. Sintaxe do continue: `continue;`



Exercícios Propostos III

1. Uma empresa deseja aumentar seus preços em 20%. Faça um programa que leia o código e o preço de custo de cada produto e calcule o preço novo. Calcule também, a média dos preços com e sem aumento. Mostre o código e o preço novo de cada produto e, no final, as médias. A quantidade de dados que serão cadastrados é desconhecida.
2. Uma loja tem 100 clientes cadastrados e deseja mandar uma correspondência a cada um deles anunciando um bônus especial. Escreva um programa que leia o código do cliente e o valor das suas compras no ano passado e calcule um bônus de 10% se o valor das compras for menor que R\$ 5000 e de 15 %, caso contrário.
3. Chico tem 1,50 metro e cresce 2 centímetros por ano, enquanto Zé tem 1,10 metro e cresce 3 centímetros por ano. Construa um programa que calcule e imprima quantos anos serão necessários para que Zé seja maior que Chico.
4. Escrever um programa que leia 5 pares de valores, o primeiro valor é a matrícula do aluno, e o segundo a sua altura em centímetros. Encontre o aluno mais alto e o mais baixo. Mostre a matrícula do aluno mais alto e do mais baixo, junto com suas alturas.
5. Escrever um programa que leia um conjunto de 10 informações contendo, cada uma delas, a altura e o sexo(1- mas/2- fem), calcule e mostre o seguinte:
 - a) a maior e a menor altura da turma
 - b) a média da altura das mulheres
 - c) a média da altura da turma.
6. Foi feita uma pesquisa entre os habitantes de uma região. Foram coletados os dados de idade, sexo (1- mas/2-fem) e salário. Faça um programa que informe:
 - a) a média de salário do grupo;
 - b) maior e menor idade do grupo;
 - c) quantidade de mulheres com salário até R\$ 200.

7. Faça um programa que mostre os conceitos finais dos alunos de uma classe de 25 alunos, considerando:

- os dados de cada aluno (matrícula e media final) serão fornecidos pelo usuário.
- a tabela de conceitos segue abaixo:

Nota	Conceito
de 0,0 a 4,9	D
de 5,0 a 6,9	C
de 7,0 a 8,9	B
de 9,0 a 10,0	A

A quantidade de dados que serão cadastrados é desconhecida.

8. O departamento de Turismo da Prefeitura de Aracaju tem anotado para suas 15 praias os seguintes dados:

Código da Praia
 Distância do centro em Km
 Número de veranistas da última temporada
 Código do tipo de estrada de acesso:
 0 - não asfaltada
 1 - asfaltada

Faça um programa para descobrir:

- O número de praias que distam mais de 15 km do Centro;
- O número de veranistas que freqüentam as praias cujo acesso não é asfaltado;
- O percentual de veranistas que freqüentaram as praias que distam menos 15 Km do Centro;

9 A prefeitura de uma cidade fez uma pesquisa entre seus habitantes, coletando dados sobre o salário e número de filhos. A prefeitura deseja saber:

- média do salário da população;
- média do número de filhos;
- maior e menor salário;
- percentual de pessoas com salário até R\$ 500.

A quantidade de dados que serão cadastrados é desconhecida.

10. Elaborar um programa para efetuar um diagnóstico. A quantidade de pacientes é desconhecida. Para cada paciente serão lidos os seguintes dados:

Código do paciente
 Código p/ Infecção dos pulmões: 0 – Ausente/ 1 - Presente
 Código p/ Temperatura: 0 – Normal/1 - Anormal
 Código p/ Coriza: 0 – Ausente/ 1 - Presente
 Código p/ Espirro: 0 – Ausente/ 1 - Presente

Diagnostico:

- Se o paciente tiver infecção nos pulmões e temperatura anormal o diagnóstico é pneumonia.
- Se o paciente tiver apenas um sintoma, qualquer deles, o diagnóstico é gripe.
- Se o paciente tiver dois ou mais sintomas, que não sejam simultaneamente infecção nos pulmões e temperatura anormal, o diagnóstico é resfriado.

Para cada paciente será impresso o código com o respectivo diagnóstico.

11. Com o lançamento dos minicomputadores Laptop no mercado nacional, um fabricante deseja obter determinadas informações dos seus possíveis consumidores. Para cada empresa ou órgão pesquisado foi digitado os seguintes dados:

Código da empresa
Preferência desejada (fabricante): 1. Toshiba/2. Compaq/3. IBM
Possuem computadores de grande porte: 1. Sim/2. Não
Necessitam de minicomputadores: 1. Sim/2. Não
Deseja fazer expansão: 1.Sim/2.Não

Deseja-se saber:

- A quantidade de empresas pesquisadas;
- Quantas empresas possuem computadores de grande porte;
- Percentual de empresa que desejam fazer expansão;
- Percentual de empresas que tem preferência pela Toshiba.

A quantidade de dados que serão cadastrados é desconhecida.

12. Foi feita uma estatística nas 50 principais cidades de quatro estados para coletar dados sobre acidentes de trânsito. Foram obtidos os seguintes dados:

- Código da cidade
- Estado (1-PE, 2-AL, 3-PB, 4-SE)
- Número de veículos de passeio (em 2002)
- Número de acidentes de trânsito com vítimas (em 2002)

Deseja-se saber:

- a) qual o maior e o menor índice de acidentes de trânsito e a que cidades pertencem;
- b) qual a média de veículos nas cidades;
- c) qual a média de acidentes com vítimas entre as cidades do Estado de Pernambuco.

13. Foi feita uma pesquisa entre os 1000 habitantes de uma região para coletar os seguintes dados: sexo (1-fem/ 2-mas), idade e altura. Faça um programa que leia as informações coletadas e mostre as seguintes informações:

- a) média da idade do grupo;
- b) média da altura das mulheres;
- c) média da idade dos homens;
- d) percentual de pessoas com idade entre 18 e 35 anos (inclusive).

14. Foi realizada uma pesquisa de algumas características físicas da população de uma certa região. Foram entrevistadas 100 pessoas e coletados os seguintes dados:

- Sexo: (1-mas/2-fem);
- Cor dos olhos: (1-azuis/2-verdes/3-castanhos);
- Cor dos cabelos: (1-louros/2-castanhos/3-pretos);
- Idade

Deseja-se saber:

- a maior idade do grupo
- a quantidade de indivíduos do sexo feminino, cuja idade está entre 18 e 35 anos e que tenham olhos verdes e cabelos louros.

15. A Empresa Legal decidiu fazer um levantamento dos candidatos que se inscreveram para preenchimento de vaga no seu quadro de funcionários, utilizando processamento eletrônico e você foi contratado, então faça um programa que:

Leia um conjunto de informações para cada candidato, contendo: número de inscrição do candidato, idade, sexo, experiência anterior(1-sim/2-nao)

Calcule:

- Quantidade de candidatos
- Quantidade de candidatas
- Média de idade dos homens com experiência
- Percentagem dos homens com mais de 45 anos, entre os homens
- Quantidade de mulheres com idade inferior a 35 anos e com experiência
- Menor idade entre as mulheres que já tem experiência no serviço

A quantidade de dados que serão cadastrados é desconhecida.

16. Faça um programa para realizar a totalização dos votos da eleição para a prefeitura concorrida entre 3 candidatos. Para cada seção são informados o número de votos do candidato A, o número de votos do candidato B, o número de votos do candidato C, o número de votos brancos e o número de votos nulos. A quantidade de dados que serão cadastrados é desconhecida.

Determine e imprima:

- a. O número de votantes;
- b. O total de votos de cada candidato;
- c. O total de votos brancos e total de votos nulos;
- d. O total de votos válidos;
- e. O candidato com maior número de votos;
- f. Se a eleição foi válida e para isso o total de votos brancos mais votos nulos deve ser menor que o total de votos válidos;
- g. Se haverá segundo turno, para não haver segundo turno basta que o total de votos do candidato vencedor seja maior que 50% dos votos válidos
- h. Percentual de votos nulos.

17. Foi feita uma pesquisa para determinar o índice de mortalidade infantil em um certo período. Fazer um programa que:

- leia inicialmente a quantidade de crianças mortas;
- Em seguida, para cada criança serão lidos os seguintes dados: o sexo(1-mas/2-fem) e quantos meses a criança viveu.

Determine e imprima:

- Percentual de crianças do sexo masculino mortas no período;
- Percentual de crianças que viveram entre 10 e 15 meses;
- Quantidade de crianças do sexo feminino mortas com menos de 3 meses de vida.

18. Escrever um programa que lê 5 valores e conta quantos destes são negativos, escrevendo esta informação.

19. Construir um programa que calcule a média aritmética de vários valores inteiros positivos. A quantidade de números que serão cadastrados é desconhecida.

20. Escreva um programa que calcule a média dos números digitados pelo usuário, apenas se eles forem pares. A quantidade de números que serão cadastrados é desconhecida.
21. Escrever um programa que leia um número n que indica quantos valores devem ser lidos a seguir. Para cada número lido, mostre uma tabela contendo o valor lido e o fatorial deste valor.
22. Escrever um programa que leia um número não determinado de valores e calcule a média aritmética dos valores lidos, a quantidade de valores positivos, a quantidade de valores negativos e o percentual de valores negativos e positivos. Mostre os resultados.
23. Escrever um programa que lê um conjunto não determinado de valores, um de cada vez, e escreve o valor lido, seu quadrado e seu cubo.
24. Faça um programa que leia uma quantidade não determinada de números positivos. Calcule a quantidade de números pares e ímpares, a média de valores pares e a média geral dos números lidos.
25. Faça um programa que leia vários números inteiros e calcule o somatório dos números negativos.
26. Escrever um programa que gera e escreve os números ímpares entre 100 e 200.
27. Escreva um programa que leia 50 valores e encontre o maior e o menor deles. Mostre o resultado.
28. Escrever um programa que leia 5 valores para uma variável N e, para cada um deles, calcule a tabuada de 1 até n . Mostre a tabuada na forma:
$$\begin{array}{l} 1 \times n = n \\ 2 \times n = 2n \\ 3 \times n = 3n \\ \dots\dots \\ n \times n = n^2 \end{array}$$
29. Escrever um programa que lê um número não determinado de pares de valores M , N , todos inteiros e positivos, um par de cada vez, e calcula e escreve a soma dos n inteiros consecutivos a partir de M inclusive.

Ex: $M = 5$ e $N = 3$
Soma = 5 + 6 + 7
30. Ler 50 notas e imprimir as três maiores.

Capítulo 5 – Vetores e Registros

5.1 Vetores - Tipos de Dados Estruturados

As *variáveis compostas homogêneas*, mais conhecidas como *arrays*, correspondem a conjuntos de elementos de um mesmo tipo, representados por um único nome.

Vetor

V =

4	7	2	5	3
---	---	---	---	---

Cada elemento do array pode ser referenciado através de índices. Exemplos:

V[0] = 4

V[1] = 7

V[4] = 3

Declaração

Tipo-da-variavel Nome_Vetor[quantidade_elementos];

Ex: int v[30];
 float n[10];

Atribuição

v[i] = 10;
v[2] = 12 + x;
v[i+2] = 14;

Leitura

```
for (i=0; i<30; i++)  
{ printf("\n\nElemento %d: ",i+1);  
  scanf("%d",&v[i]);  
}
```

Impressão

```
printf("\n\nVetor: ");
for (i=0; i<30; i++)
    printf(" %d ",v[i]);
```

Exemplos:

```
#include <stdio.h>
#include <conio.h>
main()
{ int m[10];
  float n[10];
  int qa,i;
  float soma, med;
  soma = 0;
  printf("Faculdade Legal\n\n");
  printf("Quantidade de Alunos: ");
  scanf("%d",&qa);
  for (i=0; i<qa; i++)
  { printf("\n\nDados do Alunos %d", i+1);
    printf("\nMatricula: ");
    scanf("%d",&m[i]);
    printf("\nNota: ");
    scanf("%f",&n[i]);
    soma += n[i];
  }
  med = soma/qa;
  printf("\n\nMedia da turma: %.2f",med);
  printf("\n\nAlunos com nota acima da media da turma\n");
  for (i=0; i<qa; i++)
    if (n[i]>med)
      printf("%d \n",m[i]);
  printf("\n\n\tDigite qualquer tecla para sair\n\n\n ");
  getch();
}
```

```

#include <stdio.h>
#include <conio.h>
main()
{ int v1[10], v2[10],vs[10];
  int q1,q2, i;
  printf("Soma Vetores\n\n");
  printf("Quantidade de Elementos do vetor 1: ");
  scanf("%d",&q1);
  printf("\n\nVetor 1 ");
  for (i=0; i<q1; i++)
  { printf("\nElemento %d: ",i+1);
    scanf("%d",&v1[i]);
  }
  printf("\n\nQuantidade de Elementos do vetor 2: ");
  scanf("%d",&q2);
  printf("\n\nVetor 2 ");
  for (i=0; i<q2; i++)
  { printf("\nElemento %d: ",i+1);
    scanf("%d",&v2[i]);
  }
  if (q1 == q2)
  { printf("\n\nResultado\n");
    printf("\nVetor 1: ");
    for (i=0; i<q1; i++)
      printf("%d ",v1[i]);
    printf("\n\nVetor 2: ");
    for (i=0; i<q2; i++)
      printf("%d ",v2[i]);
    printf("\n\nVetor Soma: ");
    for (i=0; i<q2; i++)
    { vs[i] = v1[i] + v2[i];
      printf("%d ",vs[i]);
    }
  }
  else
    printf("\n\nOs vetores nao podem ser somados");
  printf("\n\n\tDigite qualquer tecla para sair\n\n\n ");
  getch();
}

```

5.2 Registros

São conjuntos de dados logicamente relacionados, mas de tipos diferentes (inteiro, real, char, etc.).

Exemplo: Numa dada aplicação, podem-se ter os seguintes dados sobre os alunos de uma turma:

- ⇒ Matricula
- ⇒ Nome
- ⇒ Nota1
- ⇒ Nota2
- ⇒ Media

Cada conjunto de informações do aluno pode ser referenciado por um mesmo nome, por exemplo, ALUNO. Tais estruturas são conhecidas como registros, e aos elementos do registro dá-se o nome de campos.

O conceito de registro visa facilitar o agrupamento de variáveis que não são do mesmo tipo, mas que guardam estreita relação lógica.

Declaração

```
typedef struct {  variáveis;
                  }nome_do_tipo;
```

Exemplo:

```
typedef struct { int mat;
                 char nome[20];
                 float n1, n2, med;
               }TAlunos;
```

```
TAlunos aluno;
```

Atribuição

```
Nome-do-registro.campo = valor;
```

```
aluno.n1 = 5.0;
scanf("%d", &aluno.mat);
gets(aluno.nome);
```

5.3 Vetor de Registros

Podemos ter um conjunto de registros referenciados por um mesmo nome e individualizados por índices, através da utilização de um vetor de registros.

Exemplo:

```
typedef struct { int mat;
                 char nome[20];
                 float n1, n2, med;
               }TAlunos;
```

```
TAlunos aluno[30];
```

Atribuição

```

nome-do-vetor[indice].campo = valor;

aluno[3].n1 = 5.0;
scanf("%d", &aluno[i].mat);
fflush(stdin);
gets(aluno[2].nome);

```

Exemplo:

```

for (i=0; i<qa; i++)
{
    printf("\n\nDados do Aluno %d\n\n",i+1);
    printf("\n\tNumero de Matricula do Aluno: ");
    scanf("%d",&escola[i].mat);
    printf("\n\tSerie (1-4): ");
    scanf("%d",&escola[i].serie);
    printf("\n\tTem irmao na escola (1-sim, 2-nao)? ");
    scanf("%d",&escola[i].irmao);
    switch(escola[i].serie)
    {
        case 1: escola[i].mens = 110; break;
        case 2: escola[i].mens = 130; break;
        case 3: escola[i].mens = 160; break;
        case 4: escola[i].mens = 170; break;
    }
    if (escola[i].irmao == 1)
        escola[i].mens = escola[i].mens*0.8;
}

```

Exercícios Propostos IV

1. Ler um conjunto de números e imprimi-los na ordem inversa da leitura. A quantidade de números também será lida e será no máximo 20.
2. Ler o tamanho e os elementos de dois vetores e em seguida, caso os vetores tenham o mesmo tamanho, gere e imprima um vetor SOMA, onde seus elementos serão formados pela soma dos elementos de mesmos índices dos dois vetores lidos
3. Ler um vetor de números e imprimir os números das posições PARES e em seguida os números das posições ÍMPARES.
4. Ler dois vetores e caso tenham tamanhos iguais armazene seus elementos alternadamente em um terceiro vetor.
5. Ler um vetor de números inteiros e imprimir as posições do maior e do menor elemento do vetor. Assuma que não existem elementos repetidos no vetor.
6. Ler um conjunto de matrícula e notas dos alunos de uma turma e imprima a matricula dos alunos de tiveram nota acima da média das notas da turma.

7. Ler um vetor de números e inverter a ordem dos elementos desse vetor no próprio vetor.
8. Ler um vetor de números e em seguida ler um conjunto de números(flag=0) e para cada numero lido imprimir o número de ocorrências deste no vetor.
9. Ler um vetor de números e ler um conjunto de números (flag = 0) e para cada número lido imprimir se este existe ou não no vetor.
10. Ler dois vetores de números e gerar um terceiro vetor formado pela INTERSECÇÃO dos dois vetores lidos.
11. Ler dois vetores de números e gerar um terceiro vetor formado pela DIFERENÇA dos dois vetores lidos.
12. Ler dois vetores de números e gerar um terceiro vetor formado pela UNIÃO dos dois vetores lidos.
13. Ler um vetor e gerar um segundo vetor com os elementos do vetor lido menos os elementos repetidos.
Ex : VetorLido = 5 1 2 7 5 3 7 5 9 3; VetorGerado = 5 1 2 7 3 9
14. Ler um vetor de números e imprimir os números que se repetem nesse vetor.
15. Ler um conjunto N de números e imprimir cada elemento do conjunto e quantas vezes este se repete.

Números Lidos

3	8	5	3	8	7	3	7	3	7
---	---	---	---	---	---	---	---	---	---

Resultado

3 - 4

8 - 2

5 - 1

7 - 3

16. Faça um programa para calcular o valor das mensalidades dos alunos. A escola possui 50 alunos e para cada um deles serão informados os seguintes dados: matrícula, nome, série(1-4), se tem irmão na escola (1-sim, 2-não). O valor da mensalidade será calculado pelo programa e depende da série do aluno e se tem irmão na escola. O aluno receberá 20% de desconto na mensalidade se tiver irmão na escola.

Mensalidade: 1ª. Serie: R\$110, 2ª. Serie: R\$130, 3ª. Serie: R\$160, 4ª. Serie: R\$170.

Armazenar os dados em um vetor de registro e imprimir os dados de todos os alunos em forma de tabela.

17. Fazer um programa para cadastrar os produtos de uma farmácia usando vetor de registro. Cada produto possui os seguintes dados: Código, nome, Tipo (1-Genérico, 2- Não Genérico), Laboratório(1-Bayer, 2- LAFAPE, 3-Pfizer), Preço de Custo, Preço de Venda. O preço de venda é calculado pelo programa e depende do tipo do produto e do laboratório. O preço de venda será o preço de custo acrescido do percentual de lucro de acordo com a tabela abaixo:

	Bayer	LAFAPE	Pfizer
Genérico	10%	15%	20%
Não Genérico	20%	35%	45%

A quantidade de produtos é desconhecida. Imprimir todos os dados dos produtos da farmácia em forma de tabela.

18. A Secretaria de Educação quer um programa para fazer um relatório sobre as escolas da cidade de Recife. Para cada escola são informados os seguintes dados: código da escola, nome, total de alunos

matriculados, total de professores, se possui laboratório de informática (1-sim ou 1-não), se possui laboratório de ciências (1-sim ou 2-não). A secretaria quer saber quanto deve ser liberado de verba para cada escola. O calculo será feito da seguinte forma:

- Para cada aluno da escola são liberados: R\$ 50
- Para cada professor são liberados: R\$ 400
- Se a Escola não tem laboratório de Informática são liberados: R\$ 10.000
- Se a Escola não tem laboratório de Ciências são liberados: R\$ 7.000

Ex: uma escola com 200 alunos, com 15 professores e tem laboratório de informática, mas não tem laboratório de ciências: Verba = $200 \times 50 + 400 \times 15 + 7000 = 10000 + 6000 + 7000 = 23.000$

Imprimir um relatório contendo todos os dados das escolas e o valor que cada uma receberá da secretária de educação. Também deve ser impresso o seguinte relatório:

- Quantidade Total de Alunos nas Escolas
- Quantidade de escolas sem laboratórios de Informática
- Quantidade de escolas com os dois laboratórios (informática E de ciências)
- Total de verba liberada pela secretaria

OBS: A quantidade de escola é desconhecida.

19. Fazer um programa para a companhia de energia elétrica da cidade. Para cada consumidor são fornecidos os seguintes dados: matrícula do consumidor, nome, tipo do consumidor (1-comercial, 2-industrial, 3-residencial), quantidade de KWh consumidos no mês. O programa irá calcular o valor da conta. O programa irá ler uma quantidade de consumidores e armazenará os dados dos consumidores em um vetor de registro. Imprimir um relatório contendo os dados do consumidor e o valor que o mesmo deve pagar e no final, imprimir o relatório estatístico:

- Total a ser arrecadado pela empresa de energia
- Média de consumo das indústrias
- Total de consumo das residências
- Quantidade de estabelecimentos comerciais

OBS: Valor de 1 KWh

- Residência: R\$ 5,00
- Comércio: R\$ 3,00
- Indústria: R\$ 2,00

20. Fazer um programa para um hotel que possui 20 quarto, 15 tipo Standard e 5 tipo Luxo. Para cada hóspede serão lidos os seguintes dados: Código, Nome, Região de Origem (1-Nordeste, 2-Sudeste, 3-Sul, 4-Outros), Motivo da Viagem (1-lazer, 2-trabalho, 3-outros), Sexo, Tipo de Quarto Desejado(1-Standard, 2-Luxo) e Quantidade de dias. O programa irá calcular o valor pago pelo hospede. No final deve ser impresso o relatório com os dados de todos os hóspedes, além do relatório estatístico.

O cálculo do valor a ser pago pelo hóspede segue as regras abaixo:

- Se o quarto for do tipo 1, a diária custa R\$ 100,00.
- Se o quarto for do tipo 2 a diária custa R\$ 130,00.

- Relatório Final:
 - Quantidade de homens em quarto de luxo
 - percentual de homens que viajam a trabalho
 - quantidades de mulheres provenientes da região Sudeste
 - total arrecadado pelo hotel

A quantidade de hospede é desconhecida.

OBS: Verificar se o tipo de quarto que o hóspede deseja está disponível.

21. Faça um programa para ler 10 números inteiros, armazenando-os em um vetor. Imprimir os valores digitados e os seguintes resultados:

- A soma dos elementos das posições pares
- O produto dos elementos das posições ímpares
- O quadrado do terceiro elemento
- A diferença do sexto elemento pelo primeiro elemento
- O maior valor do vetor

22. Uma empresa de turismo deseja um programa que calcule o valor das viagens dos clientes. Serão informados os seguintes dados: código do cliente, nome, roteiro desejado(1-Brasil, 2-EUA, 3-África, 4-Japão), tipo de quarto (1-Standard, 2-Luxo), Se deseja alugar carro(1-sim, 2- não) e a quantidade de dias. A quantidade de clientes é desconhecida. O programa irá calcular o total da viagem usando os valores da tabela abaixo, que variam de acordo com o roteiro escolhido. A diária do quarto de luxo é R\$30 mais cara que o valor da diária em um quarto standard. Exemplo: Se a pessoa escolher roteiro 2 em quarto de luxo, a diária irá custar: $320 + 30 = \text{R\$ } 350$.

Roteiro	Diária – Hotel Quarto Standard	Diária - Aluguel de carro
1	R\$ 170	R\$ 50
2	R\$ 350	R\$ 60
3	R\$ 370	R\$ 75
4	R\$190	R\$ 70

Armazenar os dados em um vetor de registro. Imprimir os dados em forma de tabela, além do relatório estatístico.

Viagem Legal – Turismo	Viagem Legal – Turismo	Viagem Legal – Turismo																												
Código do Cliente: Roteiro(1-Brasil, 2-EUA, 3-África, 4-Japão): Tipo de Quarto (1- Standard, 2-Luxo): Alugar Carro(1-sim, 2-não) ? Quantidade de Dias:	Relatório Geral <table><tr><th>Cód</th><th>Nome</th><th>Rot</th><th>quarto</th><th>carro</th><th>dias</th><th>Total</th></tr><tr><td>xx</td><td>xx</td><td>xx</td><td>xx</td><td>xx</td><td>x</td><td>xx</td></tr><tr><td>xx</td><td>xx</td><td>xx</td><td>xx</td><td>xx</td><td>x</td><td>xx</td></tr><tr><td>xx</td><td>xx</td><td>xx</td><td>xx</td><td>xx</td><td>x</td><td>xx</td></tr></table> Tecle enter para relatório estatístico...	Cód	Nome	Rot	quarto	carro	dias	Total	xx	xx	xx	xx	xx	x	xx	xx	xx	xx	xx	xx	x	xx	xx	xx	xx	xx	xx	x	xx	Relatório Estatístico Quant. de clientes estão em quarto de luxo: Percentual de clientes que viajam para o Japão: Média dos valores das viagens para o Brasil: Valor da viagem mais barata: Tecle enter para sair...
Cód	Nome	Rot	quarto	carro	dias	Total																								
xx	xx	xx	xx	xx	x	xx																								
xx	xx	xx	xx	xx	x	xx																								
xx	xx	xx	xx	xx	x	xx																								
Inserir outro(1-sim,2-não)?																														

Total da Viagem: dias*diáriahotel + dias*diariacarro(se o cliente for alugar carro)

23. Faça um programa para uma empresa de celular contendo as seguintes operações:

Cadastro	Relatório Geral
Ligue Mais	Ligue Mais
Cadastro de Cliente	Listagem Geral
Código:	Codigo Nome Idade Sexo Quant. Lig. Plano Total
Nome:	
Idade:	
Sexo(1-fem/2-mas):	
Quantidade de Ligações:	
Plano(1-Dia/2-Noite/3-Fixo/4-Empresarial):	
Cadastrar outro (S/N)?	Tecla enter para relatório estatístico...

Relatório Estatístico
Ligue Mais
Relatório Estatístico
Código do cliente mais velho:
Valor da menor conta:
Media das ligações dos homens:
Plano com mais clientes:
Tecla enter para sair...

- O programa irá calcular o valor da conta de cada cliente. Sabe-se que cada plano tem uma tarifa diferenciada, segundo a tabela abaixo:

Plano	Valor de 1 ligação
Dia	1.30
Noite	1.60
Fixo	1.25
Empresarial	1.10

- Armazenar os dados em um vetor de registro. Imprimir os dados em forma de tabela, além do relatório estatístico.

24. Faça um programa para montar a folha de pagamento dos empregados de uma empresa. Para cada empregado serão lidos os seguintes dados: matrícula, nome, cargo(1-Coordenador, 2-Analista de Sistemas, 3- programador, 4- Digitador), sexo(1-mas/2-fem), anos de experiência e quantidade de filhos. O programa irá calcular para cada empregado o seu salário. Sabe-se que:

Cargo	Salário Base
1	R\$ 2500
2	R\$ 1700
3	R\$ 1400
4	R\$ 900

O salário será calculado da seguinte forma: Cada cargo tem o salário base. Além do salário base, o empregado recebe os seguintes adicionais: R\$ 50, para cada ano de experiência e R\$ 40 por cada filho. Portanto, um Coordenador com três anos de experiência e dois filhos terá o salário = $2500 + 50 \cdot 3 + 40 \cdot 2 = \text{R\$ } 2730$.

Cadastro
Empresa Legal
Cadastro de Funcionario
Matricula:
Nome:
Cargo(1-Coordenador, 2-Analista de Sistemas, 3- programador, 4- Digitador):
Sexo(1-mas/2-fem):
Anos de experiência:
Quantidade de filhos:
Cadastrar outro (1-sim/2-nao)?

Relatório Geral	Relatório Estatístico
Empresa Legal	Empresa Legal
Listagem Geral	Relatório Estatístico
Matricula Nome Cargo Sexo Anos Q. Filho Salario	Quantidade de homens analistas de sistemas:
....	Média do salário das mulheres:
....	Percentual de homens com mais de três filhos:
....	Quantidade de mulheres com mais de 10 anos de experiência:
....	Código do funcionário com maior salário:
Tecla enter para relatório estatístico...	Tecla enter para sair...

Armazenar as informações em um vetor de registros. Imprimir o relatório geral em forma de tabela contendo os dados de cada empregado. Também deve ser impresso o relatório estatístico.

25. Faça um programa para uma construtora que está vendendo apartamentos em quatro prédios. O programa deverá calcular o preço de venda e o valor da prestação do apartamento desejado pelo cliente. A quantidade de clientes é desconhecida. Armazenar os dados em um vetor de registro e apresentar os resultados em forma de tabela (com paginação, 5 por tela).

Dados de Entrada: Cliente (15 caracteres), Prédio (1-Solar, 2-Praiano, 3-Mar e Sol, 4-Costa Sul), Andar (1-6), quantidade de vagas de carro (1 ou 2) e quantidade de prestações.

Na tabela abaixo são apresentados os preços cobrados pela construtora.

Prédio	Preço Base	Adicional pelo andar	Adicional da vaga de carro
1	R\$ 160000	R\$ 17000	R\$ 2200
2	R\$ 80000	R\$ 8000	R\$ 1800
3	R\$ 135000	R\$ 11000	R\$ 1500
4	R\$ 97000	R\$ 9000	R\$ 1250

Regras para o cálculo do preço de venda: cada apartamento tem um preço base de venda, dependendo do prédio. Apartamentos do quarto, quinto e sexto andar recebem um adicional por conta do andar, conforme a tabela. Se cliente desejar 2 vagas, deve ser adicionado o preço da vaga de carro. Se ele desejar apenas uma vaga, não haverá adicional por vaga de carro. Portanto, o preço de venda será composto pelo preço base + adicional de andar (se necessário) + adicional de vaga(se necessário).

O valor da prestação = preço de venda do apartamento / quantidade de prestações.

Exemplos de telas:

Construtora Sua Casa Cliente: Prédio(1-Solar, 2-Praiano, 3-Mar e Sol, 4-Costa Sul): Andar(1-6): Quantidade de vagas de carro(1 ou 2): Quantidade de prestações: Cadastrar outro (1-sim/2-não)?	Construtora Sua Casa Relatório Geral <table border="1"> <thead> <tr> <th>Cliente</th> <th>Pred</th> <th>Andar</th> <th>Vaga</th> <th>Prest</th> <th>Valor de Venda</th> <th>Valor Prestação</th> </tr> </thead> <tbody> <tr> <td>xxxx</td> <td>x</td> <td>x</td> <td>x</td> <td>x</td> <td>xxx</td> <td>xxx</td> </tr> <tr> <td>xxxx</td> <td>x</td> <td>x</td> <td>x</td> <td>x</td> <td>xxx</td> <td>xxx</td> </tr> <tr> <td>xxxx</td> <td>x</td> <td>x</td> <td>x</td> <td>x</td> <td>xxx</td> <td>xxx</td> </tr> </tbody> </table> Página x de x...tecle enter....	Cliente	Pred	Andar	Vaga	Prest	Valor de Venda	Valor Prestação	xxxx	x	x	x	x	xxx	xxx	xxxx	x	x	x	x	xxx	xxx	xxxx	x	x	x	x	xxx	xxx
Cliente	Pred	Andar	Vaga	Prest	Valor de Venda	Valor Prestação																							
xxxx	x	x	x	x	xxx	xxx																							
xxxx	x	x	x	x	xxx	xxx																							
xxxx	x	x	x	x	xxx	xxx																							

26. Um professor quer um programa para o cálculo da média dos alunos de uma turma. Serão informadas a matrícula, nome (20 caracteres), e as três notas dos alunos. **O professor descarta a menor nota do aluno e a media é calculada com as duas maiores notas.** Calcular a média e apresentar os resultados em forma de tabela com paginação (5 alunos por tela). A quantidade de alunos é desconhecida. Armazenar os dados em um vetor de registro.

Faculdade Legal Cadastro do aluno x Matricula: Nome: Nota 1: Nota 2: Nota 3: Cadastrar outro (1-sim/2-não)?	Faculdade Legal Relatório Geral <table border="1"> <thead> <tr> <th>Matricula</th> <th>Aluno</th> <th>Nota 1</th> <th>Nota 2</th> <th>Nota 3</th> <th>Media</th> </tr> </thead> <tbody> <tr> <td>x</td> <td>xxxxxxxxxx</td> <td>x</td> <td>x</td> <td>x</td> <td>x</td> </tr> <tr> <td>x</td> <td>xxxxxxxxxx</td> <td>x</td> <td>x</td> <td>x</td> <td>x</td> </tr> <tr> <td>x</td> <td>xxxxxxxxxx</td> <td>x</td> <td>x</td> <td>x</td> <td>x</td> </tr> </tbody> </table> Página x de x...tecle enter....	Matricula	Aluno	Nota 1	Nota 2	Nota 3	Media	x	xxxxxxxxxx	x	x	x	x	x	xxxxxxxxxx	x	x	x	x	x	xxxxxxxxxx	x	x	x	x
Matricula	Aluno	Nota 1	Nota 2	Nota 3	Media																				
x	xxxxxxxxxx	x	x	x	x																				
x	xxxxxxxxxx	x	x	x	x																				
x	xxxxxxxxxx	x	x	x	x																				

O professor também deseja o relatorio estatístico abaixo:

Faculdade Legal Relatório Estatístico Média da turma: Percentual de alunos aprovados: Quantidade de alunos reprovados: Média mais baixa da turma: Matrícula do aluno com maior média: Tecla enter para sair....
--

Capítulo 6 – Modularização

6.1 Introdução

Funções servem para dividir um grande programa em diversas partes menores. Além disso, permitem que sejam utilizadas partes de programa desenvolvidas por outras pessoas, sem que se tenha acesso ao código-fonte. Como exemplo, em capítulos anteriores foi utilizada a função `printf()` sem que fossem conhecidos detalhes de sua programação.

Programas em C geralmente utilizam diversas pequenas funções, ao invés de poucas e grandes funções. Ao dividir um programa em funções, diversas vantagens são encontradas, como impedir que o programador tenha que repetir o código diversas vezes, facilitar o trabalho de encontrar erros no código.

Cada função C é na verdade uma sub-rotina que contém um ou mais comandos em C e que executa uma ou mais tarefas. Em um programa bem escrito, cada função deve executar uma tarefa. Esta função deverá possuir um nome e a lista de argumentos que receberá. As funções em C são muito semelhantes as usadas no Pascal, com a diferença que o próprio programa principal é apenas uma função que se inicia com a palavra reservada `main()`.

6.2 Declarando uma função

```
tipo nome(lista de parametros)
{
    declaração de variáveis locais;
    comando ou bloco de comandos;
}
```

tipo determina o tipo do valor de retorno (se omitido, é assumido `int`);

nome representa o nome pelo qual a função será chamada ao longo do programa;

argumentos são informações externas transmitidas para a função (podem não existir).

Todo programa é composto de funções, sendo iniciada a execução pela função de nome `main()`.

6.3 Variáveis Locais

A declaração das variáveis, em C, deve vir no início da função, antes de qualquer comando. Uma variável declarada dentro do corpo de uma função é local, ou seja, só existe dentro da função. Ao ser iniciada a função, a variável é criada. Quando a função termina, a variável é apagada, sendo liberado seu espaço ocupado na memória.

6.4 Chamando Funções

Para executar uma função, ela deve ser chamada no corpo de uma outra função (à exceção da função `main()`, que é executada no início do programa). Uma chamada de função é feita escrevendo-se o nome da função seguido dos argumentos fornecidos, entre parênteses. Se não houver argumentos, ainda assim devem ser mantidos os parênteses, para que o compilador diferencie a chamada da função de uma variável. O comando de chamada de uma função deve ser seguido de ponto-e-vírgula.

As funções só podem ser chamadas depois de terem sido declaradas. Caso sejam chamadas sem que tenham sido declaradas, um erro de compilação ocorre.

Exemplo:

```
void quadrado(int p)
{   int k;
    k = p*p;
    printf("%i",p);
}

void main( )
{   int k=25;
    quadrado(k);
    getche( );
}
```

6.5 Comando return

Serve para retornarmos um valor calculado dentro de uma função quando chamada de alguma parte do programa.

Exemplo :

```
float calc_sin(float arg)
{   float val;
    val = sin(arg);
    return val;
}
```


6.6 Argumentos

Argumentos são utilizados para transmitir informações para a função. Já foram utilizados anteriormente nas funções `printf()` e `scanf()`.

Uma função pode receber qualquer número de argumentos, sendo possível escrever uma função que não receba nenhum argumento. No caso de uma função sem argumentos pode-se escrevê-la de duas formas: deixando a lista de argumentos vazia (mantendo entretanto os parênteses) ou colocando o tipo *void* entre parênteses.

O quinto tipo existente em C, *void* (vazio, em inglês), é um tipo utilizado para representar o nada. Nenhuma variável pode ser declarada como sendo do tipo *void*. A função `main()`, já utilizada em capítulos anteriores, é um exemplo de função sem argumentos.

Exemplo:

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
typedef struct {char nome[20];
               int mat;
               }alunos;
alunos turma[30];
const int maximo= 30;
int qa,op;

void cabec()
{ system("cls");
  printf("Faculdade Legal");
}

void inserir()
{ int cont;
  do{ cabec();
    printf("\n\nInserir Novo Aluno\n\n");
    if (qa < maximo) // verifica se o vetor pode receber novo aluno
    { printf("\nNumero de Matricula do Aluno: ");
      scanf("%d",&turma[qa].mat);
      printf("\n\tNome: ");
      fflush(stdin);
      gets(turma[qa].nome);
      qa++;
      printf("\n\n\tAluno Inserido com Sucesso!!!\n\n");
    }
    else // vetor cheio
    { printf("\n\n\tNao Pode Inserir - Turma Cheia!!!\n\n");
      getch();
      break;
    }
    printf("\n\n\tContinuar inserindo aluno (1-sim/2-nao)? ");
    scanf("%d",&cont);
  }while (cont == 1);
}
```

```

void listagem()
{
    int i;
    cabec();
    printf("\n\nRelatorio Geral\n");
    printf("\n\nMatricula      Aluno\n");
    printf("\n-----");
    for(i = 0; i < qa; i++)
        printf("\n%9d      %-20s ",turma[i].mat,turma[i].nome);
    printf("\n-----\n\n");
    printf("\n\nDigite qualquer tecla para sair\n\n\n ");
    getche();
}

main()
{
    qa =0;
    do // tela com o menu do opcoes
    {
        cabec();
        printf("\n\nOpcoes:  \n\n");
        printf("          1- Inserir novo Aluno\n\n");
        printf("          2- Listagem de Alunos\n\n");
        printf("          0- Sair do Programa\n\n");
        printf("Informe a Opcao desejada: ");
        scanf("%d",&op);
        switch(op)
        {
            case 1: inserir(); break;
            case 2: listagem(); break;
            case 0: break;
            default : printf("\n\n\tVoce escolheu uma opcao que nao existe!!!!\n");
                      getche();break;
        }
    } while (op != 0);
}

```

Exercícios Propostos V



1. Faça um programa para uma escola usando módulos e vetor de registro. O programa deverá ter as seguintes telas:

Tela Principal	Tela de Cadastro
Faculdade Legal	Faculdade Legal
Opções: 1-Inserir novo aluno 2-Listagem de aluno 3-Relatório estatístico 0-Sair	Inserir Novo Aluno
Entre com a opção:	Matricula: Nome: Nota 1: Nota 2:
	Inserir outro(1-sim,2-não)?

Tela da Listagem	Tela do Relatório Estatístico
Faculdade Legal	Faculdade Legal
Listagem Geral	Relatório Estatístico
Matricula Nome Nota1 Nota2 Media	Total de Alunos:
xxx xxxx xx xx xx	Media da Turma:
xxx xxxx xx xx xx	Quantidade de Aprovados:
xxx xxxx xx xx xx	Quantidade de Reprovados:
tecle enter para voltar para o menu	tecle enter para voltar para o menu

2. O PROCON necessita de um programa para uma pesquisa de preços. Na pesquisa serão coletados os preços dos produtos em três supermercados. Usar vetor de registro(declare o vetor com 20 elementos) e modularização. Segue as telas das operações que serão realizadas pelo programa.

Tela de Menu	Tela de Cadastro de produto
PROCON - Pesquisa de Preços	PROCON - Pesquisa de Preços
Opções: 1- Cadastrar Produto 2- Listagem de Produtos 3- Relatório Estatístico 0- Sair	Cadastrar Produto
Entre com a opção:	Código do Produto: Nome do Produto: Tipo(1-alimento, 2-bebida, 3- limpeza, 4-cosmético): Preço no Supermercado 1: Preço no Supermercado 2: Preço no Supermercado 3:
	Cadastrar outro(1-sim/2-não)?

Tela de Listagem de Produtos
PROCON - Pesquisa de Preços
Listagem Geral
Codigo Nome Tipo Preco1 Preco2 Preco3 Comprar no supermercado
100 Sabao em pó 3 3.28 3.70 3.19 3
120 Arroz 1 2.17 2.45 2.22 1
130 Shampoo 4 4.33 3.97 4.15 2
140 Refrigerante 2 1.10 0.95 0.79 3
tecle enter para voltar ao menu

Tela de Relatório Estatístico
PROCON - Pesquisa de Preços
Relatório Estatístico
Quantidade de produtos de limpeza:
Media de preços dos alimentos do supermercado 2:
Percentual de produtos do tipo -cosmeticos:
Código do produto mais barato do supermercado 1:
tecle enter para sair

3. Um engenheiro precisa de um programa que irá auxiliar no cálculo da quantidade de lâmpadas necessárias para iluminar os cômodo de uma casa de maneira adequada. Para determinar o número de lâmpadas necessárias para o cômodo da residência existem normas que dão o mínimo de potência exigida

por metro quadrado. Na tabela abaixo são mostradas as potências necessárias para cada metro quadrado de cômodo:

Cômodo	Potência/m ²
1 – Quarto	20W
2 – Sala	40W
3 – Banheiro	35W
4 –Cozinha	25W
5 – Varanda	15W

Supondo que só serão utilizadas lâmpadas de 60W e que cada lâmpada normal custa R\$ 5,50 e cada lâmpada fluorescente custa R\$13,80. Faça um programa que leia o tipo do cômodo(1-Quarto/2-Sala/3-Banheiro/4-Cozinha/5-Varanda), largura do cômodo, comprimento do cômodo e o tipo da lâmpada que será usada(1-normal/2-florescente) e calcule para cada cômodo: área do cômodo, quantidade de lâmpadas necessárias para iluminar o cômodo e valor gasto com lâmpadas.

Observação: A largura, o comprimento, a área e a quantidade de lâmpadas devem ser números inteiros!

Tela principal	Tela de Inserção
Iluminar Engenharia	Iluminar Engenharia
1- Cadastro de Cômodo	Cadastro de Cômodos
2- Relatório Estatístico	Tipo do cômodo(1-Quarto/2-Sala/3-Banheiro/4-Cozinha/5-Varanda): 1
0- Sair	Largura do cômodo: 2
Entre com a opção:	Comprimento do cômodo: 4
	Tipo da lâmpada(1-normal/2-florescente): 2
	Resultado
	Área do Cômodo: 8m
	Quantidade de Lâmpadas: 3
	Valor Gasto com Lâmpadas: RS 41.40
	Inserir Outro cômodo(1-sim/2-não)?

Tela de Relatório Estatístico
Iluminar Engenharia
Relatório Estatístico
Percentual de lâmpadas fluorescente:
Maior área de cômodo:
Total geral gasto com lâmpada: RS
tecle enter para voltar ao menu

OBSERVAÇÃO: Modularizar os programas 16-26 da lista de exercício IV.

Capítulo 7 – Manipulação de Caracteres

7.1 Funções de manipulação de cadeia de caracteres: biblioteca string.h

- **strcpy(char destino, char origem)** : Copia o conteúdo de uma string para outra. Deve ser empregado para atribuição de strings, no lugar do operador normal de atribuição (=).

Parâmetros:

- destino: String que irá receber o conteúdo.
- origem: String cujo conteúdo será copiado.

Exemplo: `strcpy(nome1, nome2);`

- **strncpy(char destino, char origem, int n)**: Copia no máximo *n* caracteres de uma string para a outra. Não coloca o '\0' no final de destino, a não ser que tenha atingido o final da string origem.

Parâmetros:

- destino: String que irá receber o conteúdo.
- origem: String cujo conteúdo será copiado.
- n: Número máximo de caracteres a serem copiados.

Exemplo: `strncpy(nome1, nome2, 4);`

- **strcat(char destino, char origem)**: Concatena strings, isto é, copia o conteúdo da string origem ao final da string destino. O conteúdo anterior de destino é preservado.

Parâmetros:

- destino: String que irá receber, no seu final, o conteúdo. Ela deve ter tamanho suficiente para armazenar o conteúdo atual mais o novo.
- origem: String cujo conteúdo será acrescentado ao final da outra.

Exemplo: `strcat(nome1, nome2);`

- **int strcmp(char s1, char s2):** Compara o conteúdo de duas strings. Esta função deve ser utilizada em substituição aos operadores relacionais no caso de uso com strings.

Parâmetros:

- s1 e s2: As duas strings a serem comparadas.

Valor de retorno: A função retorna 0 se o conteúdo das strings são iguais. Retorna algum valor maior que 0 se o conteúdo de s1 é maior em ordem que s2 e um valor menor que 0 se o conteúdo de s1 é menor em ordem que s2.

Exemplo: `if ((strcmp(nome1, nome2)) == 0)`

- **int strncmp(char s1, char s2, int n):** Compara apenas um trecho do início de duas strings com tamanho especificado.

Parâmetros:

- s1 e s2: As duas strings a serem comparadas.
- n: Número de caracteres a serem comparados.

Valor de retorno: O mesmo da função strcmp.

Exemplo: `if ((strncmp(nome1, nome2, 4)) == 0)`

- **int strlen(char str):** Conta o número de caracteres armazenados em uma string, antes do '\0'.

Parâmetros:

- str: A string que terá seu tamanho calculado.

Valor de retorno: Retorna o número de caracteres da string.

Exemplo: `t = strlen(nome1);`

- **strupr (char str):** converte uma string para maiúsculas. Essa função afeta apenas os caracteres alfabéticos contidos na string, excluindo dígitos ou caracteres de pontuação.

Parâmetros:

- str: A string que terá seu conteúdo transformado para maiúsculo.

Exemplo: `strupr(nome1);`

- **strlwr (char str):** converte uma string para minúsculo.

Parâmetros:

- str: A string que terá seu conteúdo transformado para minúsculo.

Exemplo: `strlwr(nome1);`

7.2 Funções de conversão de caractere: biblioteca ctype.h

Recebem um caractere por parâmetro retornam este caractere convertido para outro formato (caixa).

- **int tolower(int c):** Converte uma letra para o formato minúsculo.
- **int toupper(int c):** Converte uma letra para o formato maiúsculo.

7.3 Funções de conversão: biblioteca stdlib.h

- **int atoi(char str):** Converte uma string para um valor numérico inteiro.

Parâmetros:

- str: String a ser convertida.

Valor de retorno: Retorna um número inteiro correspondente ao valor armazenado na string.

Exemplo: `x = atoi(mes);`

- **double atof(char str);** Converte uma string para um valor numérico em ponto flutuante.

Parâmetros:

- str: String a ser convertida.

Valor de retorno: Retorna um valor double correspondente ao conteúdo armazenado na string.

Exemplo: `m = atof(media);`

Exemplo:

```
#include <stdio.h>
#include <conio.h>
#include <string.h>
#include <stdlib.h>
/* Programa para testar as principais funcoes de manipulacao de string. */

main()
{ char s1[20], s2[20], s3[20], s4[20], s6[40], s7[40], s8[10];
  char s5[20] = "Feliz\0";
  int t, qc, vi;
  float vr;
  printf("Programa para testar as funcoes de manipulacao de caracteres\n\n");
  printf("String 1: ");
  fflush(stdin);
  gets(s1);
  // funcao que verifica o tamanho da cadeia de caracteres
  t = strlen(s1);
  printf("\n\nEsta string possui %d caracteres",t);
  getch();
  printf("\n\nString 2: ");
```

```

fflush(stdin);
gets(s2);
//funcao que verifica se as strings sao iguais por completo
printf("\n\nCompara a string1 com a string2");
if (strcmp(s1, s2) == 0)
    printf("\n\n%s eh igual a %s\n\n", s1,s2);
else
    printf("\n\n%s nao eh igual a %s\n\n", s1,s2);
getche();
//funcao que verifica se parte das strings sao iguais
printf("\n\nCompara parte da string1 com a string2\n\n");
printf("Quantos caracteres vc quer comparar? ");
scanf("%d",&qc);
if (strncmp(s1, s2,qc) == 0)
    printf("\n\nOs primeiros %d caracteres da string1 sao iguais aos da
        string2\n\n",qc);
else
    printf("\n\nOs primeiros %d caracteres da string1 nao sao iguais aos da
        string2\n\n",qc);
getche();
//copiar o conteudo de uma string para outra.
printf("\n\nCopia conteudo da string2 na string3");
strcpy(s3,s2);
printf("\n\nString 3 = %s",s3);
//atribui um conteudo para a string3
printf("\n\nAtribui um conteudo para a string3\n");
strcpy(s3,"aprovado\0");
printf("\n\nString 3 = %s",s3);
getche();
//copiar parte do conteudo de uma string para outra.
printf("\n\nCopia parte conteudo da string2 na string4\n\n");
printf("Quantos caracteres vc quer copiar? ");
scanf("%d",&qc);
strncpy(s4,s2,qc);
s4[qc]='\0';
printf("\n\nString 4 = %s\n\n",s4);
getche();
printf("\n\nString 5 = %s\n\n",s5);
getche();
//Concatenacao de strings.
printf("\n\njunta conteudo da string 6 com a string 2\n\n");
printf("String 2: %s\n\n",s2);
printf("\n\nString 6: ");
fflush(stdin);
gets(s6);
strcat(s6,s2);
printf("\n\nString 6 = %s\n\n",s6);
getche();
//Concatenacao de strings de parte da string.
printf("\n\njunta conteudo da string 7 com parte do conteudo string 2\n\n");
printf("String 2: %s\n\n",s2);
printf("\n\nString 7: ");
fflush(stdin);
gets(s7);
strncat(s7,s2,3);
printf("\n\nString 7 = %s\n\n",s7);
getche();
//transformar uma string para inteiro
printf("Converter string em numero inteiro\n\n");

```



```

printf("String 8: ");
fflush(stdin);
gets(s8);
vi = atoi(s8);
printf("\n\valor em inteiro = %d\n\n",vi);
getche();
// transformar um string em real
printf("Converter string em numero real\n\n");
printf("String 8: ");
fflush(stdin);
gets(s8);
vr = atof(s8);
printf("\n\valor em real = %.2f\n\n",vr);
getche();
}

```

7.4 Funções de teste para caracteres: Biblioteca ctype.h

Todas estas funções recebem um único caractere como parâmetro e retorna verdadeiro (um valor diferente de 0) se a condição foi satisfeita, ou falso (0) em outro caso.

- **int isalnum(int c):** Testa se o caractere passado por parâmetro é uma letra ou dígito numérico.
- **int isalpha(int c):** Testa se o caractere passado por parâmetro é uma letra.
- **int isdigit(int c):** Testa se o caractere passado por parâmetro é um dígito numérico.
- **int islower(int c):** Testa se o caractere passado por parâmetro é uma letra minúscula.
- **int isupper(int c):** Testa se o caractere passado por parâmetro é uma letra maiúscula.
- **int isspace(int c):** Testa se o caractere passado por parâmetro é um espaço em branco. O caractere correspondente à barra de espaços, quebra de linha e tabulações são considerados brancos.

Exemplo:

```

#include <stdio.h>
#include <conio.h>
#include <ctype.h>
/* Programa para testar as principais funcoes de manipulacao de caracteres. */
main()
{ char s1, s2, s3, s4, s6, s7, s8;
  int tecla, aux;
  float aux2;
  printf("Programa para testar as funcoes de manipulacao de caracteres\n\n");
  printf("\n\nPressione qualquer tecla: ");
  tecla = getchar();
  if (isdigit(tecla)==0)
    printf("\n\nvc nao digitou um numero\n\n");
  else
    printf("\n\nvc digitou um numero\n\n");
  if (isalpha(tecla)==0)
    printf("\n\nvc nao digitou uma letra\n\n");
  else
    printf("\n\nvc digitou uma letra\n\n");
  if (islower(tecla)==0)
    printf("\n\nvc nao digitou uma letra minuscula\n\n");

```

```

else
    printf("\n\nvc digitou uma letra minuscula\n\n");
if (isupper(tecla)==0)
    printf("\n\nvc nao digitou uma letra maiuscula\n\n");
else
    printf("\n\nvc digitou uma letra maiuscula\n\n");
getche();
}

```

7.5 Validação de Campo

Algumas funções apresentadas acima são utilizadas para a validação de campos, verificando se o usuário esta digitando o valor com o tipo e intervalo esperado. No exemplo a seguir é feito um teste com um campo numérico e com um campo real.

```

#include <conio.h>
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>

int testemat(char matl[10])
{ int t, v, i;
  t= strlen(matl);
  v=0;
  for(i=0;i<t;i++)
  {   if (isdigit(matl[i])==0)
      {   v=1;
          break;
      }
  }
  return v;
}

int testenota(char notal[10])
{ int t, v, i, cp;
  t= strlen(notal);
  v=cp=0;
  for(i=0;i<t;i++)
  {   if ((isdigit(notal[i])==0) && (notal[i] != '.'))
      {   v=1;
          break;
      }
      if (notal[i] == '.')
      {   cp++;
          if (cp>1)
          {   v=1;
              break;
          }
      }
  }
  return v;
}

```

```

main()
{ int numero, resp;
  float n;
  char mat[10], nota[10];
  do{ system("cls");
    printf("Testar se o usuário esta digitando um numero inteiro\n\n" );
    printf("digite a matricula: ");
    fflush(stdin);
    gets(mat);
    resp=testemat(mat);
    if (resp==0)
    { numero = atoi(mat);
      printf("\n\n matricula = %d", numero);
    }
    else
      printf("\n\nA matricula eh um campo numerico");
    getche();
  }while (resp==1);
  do{ system("cls");
    printf("Testar se o usuário esta digitando um numero real\n\n" );
    printf("\n\ndigite a nota: ");
    fflush(stdin);
    gets(nota);
    resp=testenota(nota);
    if (resp==0)
    { n = atof(nota);
      printf("\n\n nota = %f", n);
    }
    else
      printf("\n\nA nota eh um campo numerico");
    getche();
  }while (resp==1);
}

```

Exercícios Propostos VI



1. Faça um programa ler uma string S e imprime o número de brancos existentes na mesma.
2. Faça um programa que ler uma string S e um valor inteiro N e imprime os N primeiros caracteres da string S.
3. Faça um programa que ler uma string S e um valor inteiro N e imprime os N últimos caracteres da string S.
4. Faça um programa que ler uma frase e determina a quantidade de cada vogal contida na mesma.
5. Faça um programa que ler uma string e a exiba invertida.

Exemplo: Entrada → MARIA BONITA
 Saída → ATINOBAIRAM

6. Faça um programa que ler uma palavra e informa se a mesma é palíndrome. Palavras palindromes: Arara, asa, osso.

7. Faça um programa que, a partir da digitação do infinitivo de um verbo regular, faça a conjugação do mesmo no presente do indicativo para as pessoas do singular e plural .

Exemplo:

Entrada →	CANTAR
Sáída →	Eu canto
	Tu cantas
	Ele canta
	Nós cantamos
	Vós cantais
	Eles cantam

8. Faça um programa para ler uma frase e uma palavra. O programa irá verificar se a palavra está na frase.

Capítulo 8 – Comandos de tela

8.1 Comandos Básicos de Tela

Para que estes comandos funcionem no DEV-C++, devemos baixar a biblioteca conio2.

Como baixar a biblioteca conio 2

- 1) No DEV, entre na opção Ferramentas> Atualizações ... vai abrir uma janela, onde tem *select derpack*. Seleciona *derpacks.org Community derpacks* e clica em *check for updates...* agora é só procurar por CONIO e mandar atualizar, clicando em *download selected* depois de ter selecionado conio .
- 2) Se vc prestar atenção ele instala um conio 2.0 ... é uma nova biblioteca conio2.h.
- 3) clique no menu ferramentas> opções do compilador
- 4) na caixa “adicionar esses comandos à linha de comando do linker”: marque esse item e digite na caixa de edição: -lconio.
- 5) clique OK e agora pode testar com o `#include <conio2.h>`.

Comandos Básicos

Todos esses comandos estão localizados na biblioteca conio2.h

CLRSCR – Procedimento que limpa a tela e automaticamente coloca o cursor no canto superior esquerdo da mesma. Sua sintaxe é:

```
clrscr();
```

GOTOXY – Este procedimento posiciona o cursor em um ponto determinado da tela, referenciado pelos eixos X e Y, ou seja, coluna e linha. Sua sintaxe é:

```
gotoxy (x, y);
```

WHEREX – Função que retorna a coluna em que se encontra o cursor (em relação à tela atual). Sua sintaxe é:

```
X= wherex();
```

WHEREY – Função que retorna a linha em que se encontra o cursor (em relação à tela atual). Sua sintaxe é:

```
Y = wherey();
```

CLREOL – Procedimento que apaga todos os caracteres de uma linha do vídeo, que se encontram à direita do cursor. Sua sintaxe é:

```
clreol();
```

TEXTCOLOR – Procedimento que determina a cor do texto que aparecerá no vídeo. Sua sintaxe é:

```
textcolor (cor)
```

Além destas 16 cores disponíveis, podemos somar a qualquer uma delas 128 para que o texto fique piscante.

TEXTBACKGROUND – Procedimento que permite selecionar a cor de fundo da tela. Sua sintaxe é:

```
textbackground (cor);
```

As cores são: BLACK, BLUE, GREEN, CYAN, RED, MAGENTA, BROWN, LIGHTGRAY, DARKGRAY, LIGHTBLUE, LIGHTGREEN, LIGHTCYAN, LIGHTRED, LIGHTMAGENTA, YELLOW, WHITE.

DELLINE – Procedimento para deletar uma linha da tela. Remover a linha onde o cursor está posicionado. Sua Sintaxe é:

```
delline();
```

INLINE – Procedimento para inserir uma linha da tela. Insere a linha onde o cursor está posicionado. Sua Sintaxe é:

```
insline();
```

exemplo:

```
#include <stdio.h>
#include <conio2.h>
#include <stdlib.h>
typedef struct {char nome[20];
                int mat;
                float m;
                }alunos;
alunos turma[30];
const int maximo= 30;
int i,l,qa,op;
```

```
void cabec()
{ clrscr();
  printf("Faculdade Legal");
}

//Inserir novo aluno
void inserir()
{ int cont;
  do{ cabec();
    gotoxy(1,4);
    printf("Inserir Novo Aluno");
    gotoxy(1,4);
    for (i=1;i<=80;i++)
    printf("_");
    gotoxy(5,7);
    printf("Matricula do Aluno: ");
    gotoxy(5,9);
    printf("Nome do Aluno.....: ");
    gotoxy(5,11);
    printf("Media do Aluno.....: ");
    gotoxy(1,14);
    for (i=1;i<=80;i++)
    printf("_");
    gotoxy(1,17);
    for (i=1;i<=80;i++)
    printf("_");
    if (qa < maximo)
    { textcolor(YELLOW);
      gotoxy(25,7);
      scanf("%d",&turma[qa].mat);
      gotoxy(25,9);
      fflush(stdin);
      gets(turma[qa].nome);
      gotoxy(25,11);
      scanf("%f",&turma[qa].m);
      qa++;
      gotoxy(1,16);
      textcolor(RED);
      printf("Mensagem: Aluno Inserido com Sucesso!!");
      textcolor(WHITE);
    }
    else
    { gotoxy(1,16);
      textcolor(RED);
      printf("\aMensagem: Nao Pode Inserir - Turma Cheia!!");
      textcolor(WHITE);
      getche();
      break;
    }
    getche();
    gotoxy(1,16);
    clrscr();
    printf("Continuar inserindo aluno (1-sim/2-nao)? ");
    scanf("%d",&cont);
  }while (cont == 1);
}
```

```
//Imprimir relatorio com as informacoes de todos alunos
```

```
void listagem()
{ int i;
  cabec();
  gotoxy(1,4);
  printf("Relatorio Geral");
  gotoxy(1,6);
  for (i=1;i<=80;i++)
    printf("_");
  gotoxy(1,7);
  printf("Matricula");
  gotoxy(11,7);
  printf("|");
  gotoxy(13,7);
  printf("Nome ");
  gotoxy(36,7);
  printf("|");
  gotoxy(38,7);
  printf("Media");
  gotoxy(1,8);
  for (i=1;i<=80;i++)
    printf("_");
  l= 9;
  for(i = 0; i < qa; i++)
  { gotoxy(1,1);
    printf("%9d",turma[i].mat);
    gotoxy(11,1);
    printf("|");
    gotoxy(13,1);
    printf("%-20s",turma[i].nome);
    gotoxy(36,1);
    printf("|");
    gotoxy(38,1);
    printf("%5.1f",turma[i].m);
    l++;
  }
  gotoxy(1,1);
  for (i=1;i<=80;i++)
    printf("_");
  printf("\n\nDigite qualquer tecla para sair");
  getch();
}
```

```
//Programa principal
```

```
main()
{ qa =0;
  do // tela com o menu do opcoes
  { textbackground(BLACK);
    cabec();
    printf("\n\nOpcoes: \n\n");
    printf("      1- Inserir novo Aluno\n\n");
    printf("      2- Listagem de Alunos\n\n");
    printf("      0- Sair do Programa\n\n");
    for (i=1;i<=80;i++)
      printf("_");
    printf("\n");
    textcolor(YELLOW);
    printf("Informe a opcao desejada: ");
    l= wherey();
```



```

    textcolor(WHITE);
    printf("\n");
    for (i=1;i<=80;i++)
        printf(" ");
    gotoxy(27,1);
    scanf("%d",&op);
    switch(op)
    { case 1: inserir(); break;
      case 2: listagem(); break;
      case 0: break;
      default : gotoxy(27,1);
                textcolor(RED);
                printf("\aVoce escolheu uma opcao que nao existe!!");
                textcolor(WHITE);
                getch();
                break;
    }
} while (op != 0);
}
    
```



Exercícios Propostos VII

1. Fazer um programa para uma loja de confecções usando **vetor de registro, modularização e comando de tela**. O programa irá fazer o controle dos produtos da loja. Cada produto possui os seguintes dados: código, descrição, preço de venda e quantidade em estoque.

Tela Principal	Tela de Inserir Produto
Loja de Departamento Santa Maria	Loja de Departamento Santa Maria
Opções: 1. Cadastrar Produto 2. Remover Produto 3. Consultar Produto por Código 4. Alterar Dados do Produto 5. Listagem de Todos os Produtos 0.Sair	Cadastro de Produto Código: Descrição: Preço de Venda: Quantidade em Estoque: Inserir outro(1-sim/2-não)?
Entre com a opção:	

Tela de Excluir Produto	Tela da consulta por Código																
Loja de Departamento Santa Maria	Loja de Departamento Santa Maria																
Excluir Produto	Consultar Produto por Código																
Código do produto a ser removido:	Código do produto:																
<table><tr><td>Código</td><td>Descrição</td><td>Preço Venda</td><td>Q. Estoque</td></tr><tr><td>xxxx</td><td>xxxxxxx</td><td>xxx</td><td>xxx</td></tr></table>	Código	Descrição	Preço Venda	Q. Estoque	xxxx	xxxxxxx	xxx	xxx	<table><tr><td>Código</td><td>Descrição</td><td>Preço Venda</td><td>Q. Estoque</td></tr><tr><td>xxxx</td><td>xxxxxxx</td><td>xxx</td><td>xxx</td></tr></table>	Código	Descrição	Preço Venda	Q. Estoque	xxxx	xxxxxxx	xxx	xxx
Código	Descrição	Preço Venda	Q. Estoque														
xxxx	xxxxxxx	xxx	xxx														
Código	Descrição	Preço Venda	Q. Estoque														
xxxx	xxxxxxx	xxx	xxx														
Confirma remoção(1-sim/2-não)?	Consultar outro(1-sim/2-não)?																
Remover outro(1-sim/2-não)?																	

Tela da Alteração de Dados	Tela da Listagem de Todos os Produtos
Loja de Departamento Santa Maria	Loja de Departamento Santa Maria
Alterar Dados do Produto	Listagem Geral
Código do produto:	Código Descrição Preço Venda Q. Estoque
Código Descrição Preço Venda Q. Estoque	xxxx xxxxxxxx xxx xxx
xxxx xxxxxxxx xxx xxx	xxxx xxxxxxxx xxx xxx
Alterar (1-Descrição/2-Preço de Venda/3-Q. Estoque/0-Sair):	pagina 1 de x...tecle enter...
Alterar outro(1-sim/2-não)?	

Módulos

1. Inserir produto: Os dados dos produtos são armazenados em um vetor de registro. A tela deve vir toda pronta, com o curso posicionando ao lado do campo que será digitado.
2. Remover Produto: informar a código do produto que deseja remover. Mostrar os dados do produto para o usuário confirmar se deseja remover.
3. Consultar produto por código: informa o código do produto que deseja consultar e se o produto estiver cadastrado, mostra os dados do mesmo.
4. Alterar dados do Produto: é informado o código do produto que deseja fazer a alteração, o registro do produto é mostrado na tela e todos os dados podem ser alterados, exceto o código.
5. Listagem Geral: irá imprimir os dados de todos os produtos em forma de tabela e com paginação(5 produtos por tela).

OBSERVAÇÃO: Para treinar mais o uso de comandos de tela, aplique-os nos programas da Lista de Exercícios V.

Capítulo 9 – Arquivo

9.1 Alocação Dinâmica - Ponteiros

Ponteiros são variáveis que armazenam um endereço de memória. Se uma variável contém o endereço de outra, então a primeira (o ponteiro) aponta para a segunda.

```
int a,*x;  
x = &a;
```

O “*” significa que esta sendo declarado o ponteiro x.

O ponteiro "x" aponta para o "inteiro" a, ou seja, ele armazena o endereço de memória da variável a.

Operadores

&- (E comercial) fornece o endereço de determinada variável. Atribui o endereço de uma variável para um ponteiro.

*- (Asterístico) que acessa o conteúdo de uma variável, cujo endereço é o valor do ponteiro. Não confundir com o operador aritmético de multiplicação de mesmo símbolo. Devolve o valor endereçado pelo ponteiro.

```
main()  
{ int *pont, cont, valor;  
  cont = 100;  
  pont = &cont;  
  valor = *pont;  
  printf("%d",valor);    /* 100 */  
}
```

```
main()  
{ int x,y,*px,*py;  
  x = 100;  
  px = &x;    // px tem o endereço de x  
  py = px;    // py tem o endereço de x  
  y = *py;    // y = 100, pois recebe o valor de x através de py  
  printf("%d %d",x,y);  
}
```

Simule a execução dos programas abaixo:

```
#include "stdio.h"  
main()  
{ int i,k,*pi,*pk;
```

```

    i = 2; k = 0;
    pk = &k;
    pi = &i;
    *pk = i;      // variável apontada por pk recebe o valor de i
    printf("para *pk = i, temos k= %d\n",k);
    k = *pi;
    printf("para k = *pi, temos k= %d\n",k);
    scanf("%c",&a);
}

main()
{   int x,y,*px,*py;
    printf("Digite um valor: ");
    scanf("%d",&x);
    px = &x;
    y = *px;
    printf("digitou= %d e y= %d\n",x,y);
    *px = 8;
    printf("valor mudou para %d\n",x);
}

```

Funções malloc e free

As funções malloc() e free() permitem que utilizemos a área de memória livre para criar nossas variáveis. As variáveis serão criadas em tempo de execução.

A função malloc() aloca memória e tem esse protótipo:

```
ponteiro = malloc(numero_de_bytes);
```

- O numero_de_bytes é a quantidade de memória que deseja alocar;
- A função malloc() retorna um ponteiro para a primeira área livre;
- A função malloc faz parte a biblioteca stdlib.h

```
int *p;
p = malloc(10000);
```

A função free() desaloca memória e tem esse protótipo:

```
free(ponteiro);
```

Ponteiro para registros

```
typedef struct {   int mat;
                  char nome[15];
                } TAluno;
```

Declaração do ponteiro para registro

```
TAluno *pa1 = NULL, *pa2 = NULL;
```

Alocação de memória para um registro

```
pa2 = (TAluno *)malloc(sizeof(TAluno));
```

Atribuição a um campo do registro

```
pa2->mat = 3;  
  
for (i=0;i<=14;i++)  
    pa2->nome[i] = nome1[i];
```

Liberar memória de endereço pa2

```
free(pa2);
```

9.2 Arquivos

Quando desejamos armazenar os dados de forma definitiva, sem que os mesmos sejam perdidos no final da execução do programa, podemos armazená-los em arquivos.

9.2.1 Declaração de um Arquivo

O sistema de entrada e saída do C é composto por uma série de funções, cujos protótipos estão reunidos em `stdio.h`. Todas estas funções trabalham com o conceito de "ponteiro de arquivo". Este não é um tipo propriamente dito, mas uma definição usando o comando `typedef`. Esta definição também está no arquivo `stdio.h`. Podemos declarar um ponteiro de arquivo da seguinte maneira:

```
FILE *p;
```

p será então um ponteiro para um arquivo.

9.2.2 Abrindo e Fechando um Arquivo

- **fopen**

Esta é a função de abertura de arquivos. Seu protótipo é:

```
FILE *fopen (char *nome_do_arquivo, char *modo);
```

- O `nome_do_arquivo` determina qual arquivo deverá ser aberto.
- O modo de abertura diz que tipo de uso você vai fazer do arquivo.

A tabela abaixo mostra os valores de modo válidos:

Modo	Significado
"r"	Abrir um arquivo texto para leitura. O arquivo deve existir antes de ser aberto.
"w"	Abrir um arquivo texto para escrita. Se o arquivo não existir, ele será criado. Se já existir, o conteúdo anterior será destruído.
"a"	Abrir um arquivo texto para escrita. Os dados serão adicionados no fim do arquivo ("append"), se ele já existir, ou um novo arquivo será criado, no caso de arquivo não existente anteriormente.
"rb"	Abrir um arquivo binário para leitura. Igual ao modo "r".
"wb"	Cria um arquivo binário para escrita, como no modo "w".
"ab"	Acrescenta dados binários no fim do arquivo, como no modo "a".
"r+"	Abrir um arquivo texto para leitura e escrita. O arquivo deve existir e pode ser modificado.
"w+"	Cria um arquivo texto para leitura e escrita. Se o arquivo existir, o conteúdo anterior será destruído. Se não existir, será criado.
"a+"	Abrir um arquivo texto para leitura e escrita. Os dados serão adicionados no fim do arquivo se ele já existir, ou um novo arquivo será criado, no caso de arquivo não existente anteriormente.
"r+b"	Abrir um arquivo binário para leitura e escrita. O mesmo que "r+" acima, só que o arquivo é binário.
"w+b"	Cria um arquivo binário para leitura e escrita. O mesmo que "w+" acima, só que o arquivo é binário.
"a+b"	Acrescenta dados ou cria um arquivo binário para leitura e escrita. O mesmo que "a+" acima, só que o arquivo é binário

Poderíamos então, para abrir um arquivo binário para escrita, escrever:

```
FILE *fp;
fp = fopen("exemplo.bin", "wb");
if (!fp)
    printf("Erro na abertura do arquivo.");
```

A condição !fp testa se o arquivo foi aberto com sucesso porque no caso de um erro a função fopen() retorna um ponteiro nulo (NULL).

Uma vez aberto um arquivo, vamos poder ler ou escrever nele utilizando as funções adequadas.

- **fclose**

Quando acabamos de usar um arquivo que abrimos, devemos fechá-lo. Para tanto usamos a função fclose():

```
int fclose(FILE *fp);

fclose(fp);
```

O ponteiro fp passado à função fclose() determina o arquivo a ser fechado. A função retorna zero no caso de sucesso.

- **feof**

EOF ("End of file") indica o fim de um arquivo. Às vezes, é necessário verificar se um arquivo chegou ao fim. Para isto podemos usar a função `feof()`. Ela retorna não-zero se o arquivo chegou ao EOF, caso contrário retorna zero. Seu protótipo é:

```
int feof (FILE *fp);  
  
while (!feof(pf))
```

9.2.3 Ler e Gravar Informação no Arquivo

Podemos escrever e ler blocos de dados. Para tanto, temos as funções `fread()` e `fwrite()`.

- **fread**

O protótipo de `fread()` é:

```
fread (void *buffer, int numero_de_bytes, int count, FILE *fp);
```

O *buffer* é a região de memória na qual serão armazenados os dados lidos. O *número de bytes* é o tamanho da unidade a ser lida. *count* indica quantas unidades devem ser lidas. Isto significa que o número total de bytes lidos é: `numero_de_bytes*count`

```
fread(&aluno, sizeof(Talunos), 1, pf);
```

- **fwrite**

A função `fwrite()` grava informação no arquivo. Seu protótipo é:

```
fwrite(void *buffer, int numero_de_bytes, int count, FILE *fp);
```

```
fwrite(&aluno, sizeof(Talunos), 1, pf);
```

9.2.4 Acesso Randômico

- **fseek**

Para se fazer procuras e acessos randômicos em arquivos usa-se a função `fseek()`. Esta move a posição corrente de leitura ou escrita no arquivo de um valor especificado, a partir de um ponto especificado. Seu protótipo é:

```
int fseek (FILE *fp, long numbytes, int origem);  
  
fseek(pf, (matrem-1)*sizeof(Talunos), SEEK_SET);
```

O parâmetro *origem* determina a partir de onde os *numbytes* de movimentação serão contados. Os valores possíveis:

Nome	Valor	Significado
SEEK_SET	0	Início do arquivo
SEEK_CUR	1	Ponto corrente no arquivo
SEEK_END	2	Fim do arquivo

Tendo definido a partir de onde irá contar, numbytes determina quantos bytes de deslocamento serão dados na posição atual.

- **rewind**

Retorna a posição corrente do arquivo para o início. A função `rewind()` tem o protótipo:

```
void rewind (FILE *fp);
```

```
rewind(fp);
```

- **remove**

Apaga um arquivo especificado. Protótipo:

```
int remove (char *nome_do_arquivo);
```

```
remove("exemplo.bin");
```

- **rename**

Renomeia um arquivo especificado. Protótipo:

```
int rename (char *nome_do_arquivo, char *nome_do_arquivo_novo);
```

```
rename("exemplo.bin", "exemplonovo.bin");
```

Programa utilizando arquivos

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
/* Programa para executar funcoes diretamente do arquivo de acesso aleatorio.
   O programa irá fazer o controle dos dados dos alunos de uma turma.
*/
typedef struct aluno{ int mat;
                      char nome[21];
                      }alunos;

alunos aa,an, aaux;
int op, qa, posicao;
FILE *pf; //ponteiro para arquivo
```



```

// Abre o arquivo. se o arquivo nao existir, ele inicializa.
void abre_arquivo()
{
    int i;
    if ((pf = fopen("arq","r+b"))== NULL)
        pf = fopen("arq","w+b"); // arquivo nao existe, precisa ser criado
    printf("\n\nArquivo pronto para ser usado");
    getche();
}

void cabec()
{
    system("cls");
    printf("Faculdade Legal\n\n");
}

int procura(int matp)
{
    int p;
    p= 0;
    fseek(pf,0, SEEK_SET);
    fread(&aa, sizeof(alunos), 1, pf);
    while ((!feof(pf)) && (aa.mat!=matp))
    {
        fread(&aa, sizeof(alunos),1, pf);
        p++;
    }
    if (aa.mat == matp)
        return p;
    else
        return -1;
}

// Inserir um novo aluno no arquivo
void inserir()
{
    int continuar;
    do{ cabec();
        printf("\nInserir Novo Aluno\n\n");
        printf("\n\tNumero de Matricula do Aluno: ");
        scanf("%d",&aa.mat);
        fflush(stdin);
        printf("\n\tNome: ");
        gets(aa.nome);
        fflush(stdin);
        fseek(pf,0,SEEK_END);
        fwrite(&aa,sizeof(alunos),1,pf);
        printf("\n\n\tAluno Inserido com Sucesso!!!\n\n");
        printf("\n\n\tContinuar inserindo aluno (1-sim/2-nao)? ");
        scanf("%d",&continuar);
    }while (continuar == 1);
}

// Remover um aluno do arquivo
void remover()
{
    int matrem, continuar, conrem;
    do{ cabec();
        printf("\nRemover Aluno\n\n");
        printf("\n\tNumero de Matricula do Aluno: ");
        scanf("%d",&matrem);
        posicao = procura(matrem);
        if (posicao == -1)
            printf("\n\n\tNumero de Matricula Incorreto!!!!!!\n\n");
    }
}

```

```

else
{ printf("\n\nMat    Aluno");
  printf("\n-----");
  printf("\n%3d    %-15s", aa.mat, aa.nome);
  printf("\n-----\n\n");
  printf("\n\tDeseja remover o aluno (1-sim/2-nao)? ");
  scanf("%d",&conrem);
  if (conrem == 1)
  { fseek(pf,posicao*sizeof(alunos), SEEK_SET);
    fwrite(&an, sizeof(alunos), 1, pf);
    printf("\n\n\tAluno removido com Sucesso!!!\n\n");
  }
  else
    printf("\n\n\tO aluno nao foi removido!!!\n\n");
}
printf("\n\n\tContinuar removendo aluno (1-sim/2-nao)? ");
scanf("%d",&continuar);
}while (continuar == 1);
}

//Consultar um aluno do arquivo
void consultar()
{ int matcon, continuar;
  do{ cabec();
    printf("\nConsultar Aluno\n\n");
    printf("\n\tNumero de Matricula do Aluno: ");
    scanf("%d",&matcon);
    posicao = procura(matcon);
    if (posicao == -1)
      printf("\n\n\tNumero de Matricula Incorreto!!!!!!\n\n");
    else
    { printf("\n\nMat    Aluno");
      printf("\n-----");
      printf("\n%3d    %-15s ", aa.mat, aa.nome);
      printf("\n-----\n\n");
    }
    printf("\n\n\tContinuar consultando aluno(1-sim/2-nao)? ");
    scanf("%d",&continuar);
  }while (continuar == 1);
}

//Fazer alteracao dos dados de um aluno
void alterar()
{ int matalt, conalt, continuar;
  do{ cabec();
    printf("\nAlterar Dados do Aluno\n\n");
    printf("\n\tNumero de Matricula do Aluno: ");
    scanf("%d",&matalt);
    posicao = procura(matalt);
    if (posicao == -1)
      printf("\n\n\tNumero de Matricula Incorreto!!!!!!\n\n");
    else
    { printf("\n\nMat    Aluno");
      printf("\n-----");
      printf("\n%3d    %-15s ", aa.mat, aa.nome);
      printf("\n-----\n\n");
      printf("\nDeseja Alterar o nome do aluno (1-sim/2-nao)? ");
      scanf("%d",&conalt);
    }
  }
}

```

```

        if (conalt == 1)
        {
            fflush(stdin);
            printf("\nNome : ");
            gets(aa.nome);
        }
        fseek(pf, posicao * sizeof(alunos), SEEK_SET);
        fwrite(&aa, sizeof(alunos), 1, pf);
        printf("\n\nMat    Aluno");
        printf("\n-----");
        printf("\n%3d    %-15s", aa.mat, aa.nome);
        printf("\n-----\n\n");
        printf("\n\nAluno Alterado com Sucesso!!!\n\n");
    }
    printf("\n\n\tContinuar alterando aluno(1-sim/2-nao)? ");
    scanf("%d", &continuar);
} while (continuar == 1);
}

//Imprimir o relatorio contendo os dados de todos os alunos
void listagem()
{
    qa = 0;
    cabec();
    printf("\nRelatorio Geral\n");
    printf("\nMat    Aluno");
    printf("\n-----");
    fseek(pf, 0, SEEK_SET);
    fread(&aa, sizeof(alunos), 1, pf);
    while (!feof(pf))
    {
        if (aa.mat != 0)
        {
            qa++;
            printf("\n%3d    %-15s", aa.mat, aa.nome);
        }
        fread(&aa, sizeof(alunos), 1, pf);
    }
    printf("\n-----");
    printf("\n\nQuantidade de alunos = %d", qa);
    printf("\n\nDigite qualquer tecla para sair\n\n\n");
    getche();
}

//Programa principal
main()
{
    abre_arquivo();
    an.mat = 0;
    do
    {
        cabec();
        printf("\n Opcoes: \n\n");
        printf("\t1- Inserir novo Aluno\n\n");
        printf("\t2- Remover Aluno\n\n");
        printf("\t3- Consultar Aluno\n\n");
        printf("\t4- Alterar dados de Aluno\n\n");
        printf("\t5- Listagem de Alunos\n\n");
        printf("\t0- Sair do Programa\n\n\n");
        printf("\t Informe a Opcao desejada: ");
        scanf("%d", &op);
        switch (op)
        {
            case 1: inserir(); break;
            case 2: remover(); break;
            case 3: consultar(); break;
        }
    }
}

```

```

        case 4: alterar(); break;
        case 5: listagem(); break;
        case 0: fclose(pf);break;
        default : printf("\n\n\t Opcao Invalida!!!!\n"); getch(); break;
    }
}while (op != 0);
}

```

Exercícios Propostos VIII



1. Faça um programa para um shopping contendo as seguintes telas:

Tela de menu	Cadastro
Shopping Center	Shopping Center
Opções: 1. Cadastrar loja 2. Consultar loja 3. Listagem de todas as lojas 0. Sair	Cadastro de Loja
Entre com a opção:	Código: Nome: Área: Tipo(1-Confecções, 2-Alimentação, 3-Livros, 4- Serviços): Cadastrar outra (Sim/Não)?

Consulta	Relatório Geral
Shopping Center	Shopping Center
Consultar Loja	Listagem Geral
Código da loja que deseja consultar:	Código Nome Área Tipo Condomínio
Código Nome Área Tipo Condomínio	
....	
....	
....	
Consultar outra (Sim/Não)?	pagina x de x... tecler enter...

- O programa irá armazenar os dados das lojas em um arquivo.
- O programa irá calcular o valor do condomínio de cada loja. O valor do condomínio depende da área e do tipo da loja. Veja tabela abaixo:

Tipo da loja	Valor do m ²
1	R\$ 50
2	R\$ 65
3	R\$ 80
4	R\$ 95

- Na listagem deve apresentar cinco lojas por tela.

OBSERVAÇÃO: Refaça os programas das Listas de Exercícios V e VII, armazenando os dados em arquivos.

Capítulo 10 – Extras

10.1 Tabela ASCII

O código ASCII (*American Standard Code for Information Interchange*) representa uma maneira de codificar caracteres na forma de valores inteiros. Neste código, os caracteres são mapeados para valores numéricos representáveis por sete dígitos binários (*bits*).

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

Source: www.LookupTables.com

128	Ç	144	É	161	í	177	⌘	193	⊥	209	〒	225	β	241	±
129	ü	145	æ	162	ó	178	⌘	194	⊥	210	〒	226	Γ	242	≥
130	é	146	Æ	163	ú	179		195	⊥	211	ℓ	227	π	243	≤
131	â	147	ô	164	ñ	180	†	196	—	212	ℓ	228	Σ	244	∫
132	ä	148	ö	165	Ñ	181	‡	197	+	213	ℓ	229	σ	245	∫
133	à	149	ò	166	²	182	‡	198	†	214	ℓ	230	μ	246	÷
134	â	150	û	167	°	183	π	199	‡	215	‡	231	τ	247	≈
135	ç	151	ù	168	¿	184	‡	200	ℓ	216	‡	232	Φ	248	°
136	ê	152	—	169	—	185	‡	201	ℓ	217	‡	233	⊙	249	.
137	ë	153	Ö	170	¬	186	‡	202	ℓ	218	‡	234	Ω	250	.
138	è	154	Ü	171	½	187	‡	203	〒	219	■	235	δ	251	√
139	ï	156	£	172	¼	188	‡	204	‡	220	■	236	∞	252	—
140	î	157	¥	173	¡	189	‡	205	=	221	■	237	φ	253	²
141	ì	158	—	174	«	190	‡	206	‡	222	■	238	ε	254	■
142	Ä	159	ƒ	175	»	191	‡	207	⊥	223	■	239	∩	255	
143	Å	160	á	176	⌘	192	⊥	208	⊥	224	α	240	≡		

Source: www.LookupTables.com

O programa abaixo, monta a tabela ascii na tela.

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>

main()
{ int n, i;
  system("cls");
  printf("Tabela ASCII\n\n");
  for (i=1; i<=255; i++)
  { printf("%6d = %2c",i, i);
    // para mudar de linha
    if (i%4==0)
      printf("\n\n");
    // para mudar de pagina
    if (i%40==0)
    { getche();
      system("cls");
      printf("Tabela ASCII\n\n");
    }
  }
  getche();
}
```

10.2 Acentos

Você já deve ter notado que quando se tem uma frase acentuada, caracteres estranhos são impressos na tela. Para colocar acento no Dev-Cpp será usada a função `CharToOem`, esta função faz com que textos no console tenham acento de modo correto. Veja o uso no exemplo a seguir.

```
#include <windows.h> // necessário para a CharToOem
#include <stdio.h>
#include <string.h>
#include <conio.h>

main ()
{
    char texto[80];
    printf("Alô maçã café não \n\n");
    strcpy(texto, "Alô maçã café não \0");
    CharToOem(texto, texto);
    printf("%s\n", texto);
    getch();
}
```

Podemos criar uma função que passará a ser utilizada toda vez que seja necessário acentuar uma frase.

```
#include <stdio.h>
#include <conio2.h>
#include <stdlib.h>
#include <windows.h>

void linha()
{
    int j;
    for (j=1; j<=80; j++)
        printf("_");
}

void aprintf(char texto[80])
{
    char text[80];
    CharToOem(texto, text);
    printf("%s", text);
}

void cabec()
{
    clrscr();
    linha();
    gotoxy(33, 2);
    aprintf("Faculdade José Gonçalves");
}

void telamenu()
{
    textbackground(BLACK);
    textcolor(LIGHTCYAN);
    cabec();
    aprintf("\n\n Opções: \n\n");
    printf("      1- Cadastrar Aluno\n\n");
    printf("      2- Listagem geral de alunos\n\n");
    aprintf("      3- Histórico escolar\n\n");
    aprintf("      4- Relatório Estatístico\n\n");
    printf("      0- Sair do Programa\n\n");
    linha();
    aprintf("\n Informe a opção desejada: ");
}

main()
{
    int op;
    do
    {
        telamenu();
        scanf("%d", &op);
    } while (op != 0);
}
```

10.3 Molduras

Podemos desenhar molduras na tela usando os símbolos especiais. Para isso, o cursor deve ser posicionado no local onde o símbolo irá aparecer e em seguida, escreve o símbolo na tela, usando o valor do símbolo(ver tabela ascii) no printf. No exemplo abaixo é montada uma moldura e a tela de menu de um programa.

```
#include<windows.h>
#include<stdio.h>
#include<conio2.h>
#include<stdlib.h>
#include<ctype.h>
#include<string.h>

char textot[80];

void retangulotriplo(int xin, int yin, int xfim, int yfim, int linhdiv1, int
linhdiv2)
{ int j, al;
  gotoxy(xin,yin);
  printf("%c",201);
  for(j=xin+1;j<xfim;j++)
    printf("%c",205);
  printf("%c",187);
  al=yfim-yin;
  for (j=1; j<=al; j++)
  { gotoxy(xin, yin+j);
    printf("%c",186);
    gotoxy(xfim, yin+j);
    printf("%c",186);
  }
  gotoxy(xin,yfim);
  printf("%c",200);
  for(j=xin+1;j<xfim;j++)
    printf("%c",205);
  printf("%c",188);
  gotoxy(xin,linhdiv1);
  printf("%c",204);
  for(j=xin+1;j<xfim;j++)
    printf("%c",205);
  gotoxy(xfim,linhdiv1);
  printf("%c",185);

  gotoxy(xin,linhdiv2);
  printf("%c",204);
  for(j=xin+1;j<xfim;j++)
    printf("%c",205);
  gotoxy(xfim,linhdiv2);
  printf("%c",185);
}
```



```
void centraliza(char texto[80], int linha)
{ int tam, c;
  tam =strlen(texto);
  c= (80-tam)/2;
  gotoxy(c,linha);
  printf("%s",texto);
}

void aprintf(char texto[80])
{ char text[80];
  int tam;
  strcpy(text,texto);
  CharToOem(text,text);
  printf("%s",text);
}

void acento(char texto[80])
{ strcpy(textot,texto);
  CharToOem(textot,textot);
}

void escreva(int x, int y, char texto[80])
{ gotoxy(x, y);
  strcpy(textot,texto);
  CharToOem(textot,textot);
  printf("%s", textot);
}

void telamenu()
{ textbackground(BLACK);
  textcolor(WHITE);
  clrscr();
  retangulotriplo(1,1,80,24,4,22);
  centraliza("FACULDADE LEGAL",2);
  acento("Sistema Acadêmico");
  centraliza(textot,3);
  escreva(5,6," Opções: ");
  escreva(10,8,"1- Cadastrar Aluno");
  escreva(10,10,"2- Listagem Geral de Alunos");
  escreva(10,12,"3- Histórico Escolar");
  escreva(10,14,"4- Relatório Estatístico");
  escreva(10,16,"0- Sair do Programa");
  escreva(2,23,"Informe a opção desejada: ");
  textcolor(YELLOW);
}

void mensagem(int x,int y, int cor, char texto[80])
{ gotoxy(x,y);
  textcolor(cor);
  strcpy(textot,texto);
  CharToOem(textot,textot);
  printf("%s", textot);
  textcolor(WHITE);
}
```

```

main()
{ int op;
  do
  { telamenu();
    scanf("%d",&op);
    switch(op)
    { case 1: break;
      case 2: break;
      case 3: break;
      case 4: break;
      case 0: break;
      default : mensagem(28,23,RED,"\aOpcao inválida...Tecle ENTER!!");
                getche(); break;
    }
  } while (op != 0);
}

```

10.4 Biblioteca time.h

Essa biblioteca tem funções que auxiliam na obtenção de data e hora do sistema. Abaixo a estrutura usada pela biblioteca.

```

struct tm
{ int tm_sec;      /* segundos: 0-59 */
  int tm_min;      /* minutos: 0-59 */
  int tm_hour;     /* horas: 0-23 */
  int tm_mday;     /* dia do mes: 1-31 */
  int tm_mon;      /* mes começa por janeiro: 0-11 */
  int tm_year;     /* ano começa por 1900 */
  int tm_wday;     /* dia da semana - começa por domingo(0-6) */
  int tm_yday;     /* dia do ano - jan. 1: 0-365 */
  int tm_isdst;    /* horario de verao */
};

```

O programa abaixo captura a data e hora do sistema e apresenta na tela.

```

#include <stdio.h>
#include <conio.h>
#include <time.h>

main()
{ time_t ltime = time(NULL);
  struct tm *hoje;//
  hoje = localtime(&ltime);
  printf("Hora e Data(biblioteca time)\n\n");
  printf("%02d:%02d:%02d\n",hoje->tm_hour,hoje->tm_min,hoje->tm_sec);
  printf("%02d/%02d/%04d\n",hoje->tm_mday,hoje->tm_mon+1,hoje->tm_year+1900);
  getche();
}

```

Capítulo 11 – Boas Práticas de Programação

11.1 Qualidades de um Programa

A correção é uma principal qualidade que deve ser considerada em um programa. Enquanto as outras qualidades são relativas, isto é, podem ser mais importantes em alguns programas e menos em outros, a correção é condição para que o programa tenha sentido de existência.

As outras qualidades podem interessar em um grau maior em algum programa e em um grau menor em outros, enquanto a correção é uma qualidade absoluta. Só interessa um programa 100% correto.

As fontes mais freqüentes de erros em programas são as falhas humanas, que se manifestam não somente na lógica incorreta ou na ausência de elementos importantes, mas também na simples transcrição errônea e em outros erros ou esquecimentos banais.

Na construção de um programa há, pois, uma hierarquia de considerações a fazer:

- **correção:** O programa deve resolver correta e completamente o problema do usuário. Fazer-se uma verificação intuitiva da correção, isto é, deve-se parar, analisar, verificar consistência, completude e adequação do que está sendo produzido. Se necessário, obter a opinião e colaboração de terceiros. Aplicar testes suficientes para garantir, pelo menos, que o programa produz os resultados esperados.
- **efetividade ou eficácia:** Diz respeito à capacidade de o programa chegar a um resultado em um tempo hábil.
- **eficiência e clareza:** Deve fazê-lo da forma mais econômica possível tanto do ponto de vista da máquina quanto do ponto de vista das pessoas que vão usá-lo. O programa mais eficiente é aquele que obtém o resultado desejado com a melhor relação custo/benefício. Clareza se refere à capacidade de o programa expressar as ações que devem ser realizadas sem que haja ambigüidades.
- **generalidade e adaptabilidade:** Servir como solução para uma classe de problemas e não somente

para um problema específico. Prever, tanto quanto possível a necessidade de futuras adaptações estando preparado para aceitá-las sem grandes dificuldades.

11.2 Formas de Melhoria da Eficiência de um Programa

A primeira preocupação com a construção de qualquer programa é construí-lo correto, pois, sem esta qualidade as outras perdem a sua razão de ser. Depois de se ter um programa correto pode-se pensar em melhorar-lhe a eficiência. Alguns cuidados na melhoria da eficiência de um programa são apresentados a seguir:

- Efetuar os cálculos que não são afetados pela repetição, fora da repetição.
- Evitar calcular a mesma expressão várias vezes sem necessidade.
- Não esgotar um “Loop” quando não for necessário.
- Não examinar condições desnecessárias.
- Escolher métodos mais eficientes quando existem várias opções.

Observação: Eficiência se refere tanto à maior velocidade do processamento quanto ao menor uso de espaço de memória. Geralmente estas duas eficiências são conflitantes. Conseguimos maior velocidade de processamento às custas de um uso de mais memória e quando tentamos economizar memória, geralmente, aumentamos o tempo de processamento.